UNIT-5

Learning

5.1 Forms of learning

- Any component of an agent can be improved by learning from data. The improvements, and the techniques used to make them, depend on four major factors:
  - ✓ Which component is to be improved.
  - ✓ What prior knowledge the agent already has.
  - ✓ What representation is used for the data and the component.
  - ✓ What feedback is available to learn from.
- Components to be learned
  - ✓ A direct mapping from conditions on the current state to actions.
  - ✓ A means to infer relevant properties of the world from the percept sequence.
  - ✓ Information about the way the world evolves and about the results of possible actions
  - ✓ the agent can take.
  - ✓ Utility information indicating the desirability of world states.
  - ✓ Action-value information indicating the desirability of actions.
  - ✓ Goals that describe classes of states whose achievement maximizes the agent's utility
- Each of these components can be learned. Consider, for example, an agent training to become a taxi driver.
  - ✓ Condition–action rules from when the instructor says "Brake!".
  - ✓ Perception by recognizing buses from labeled images.
  - ✓ Action effects by observing outcomes, like braking on wet roads.
  - ✓ Utility function from feedback, such as losing tips after rough driving.

Representation and prior knowledge

Different agent components can be represented in various ways—logical sentences, Bayesian networks, or factored attribute vectors. Learning algorithms exist for each. Most current research focuses on inductive learning, which derives general rules from specific examples, while deductive (analytical) learning derives new, logically entailed rules from existing ones for more efficient reasoning.

Feedback to learn from

There are three types of feedback that determine the three main types of learning:

- Unsupervised learning
- Reinforcement learning
- Supervised learning
- Semi-supervised learning

Unsupervised learning

- In unsupervised learning the agent learns patterns in the input even though no explicit feedback is supplied. The most common unsupervised learning task is clustering: detecting potentially useful clusters of input examples.
- For example, a taxi agent might gradually develop a concept of "good traffic days" and "bad traffic days" without ever being given labeled examples of each by a teacher.

Reinforcement learning

- In reinforcement learning the agent learns from a series of reinforcements—rewards or punishments. For example, the lack of a tip at the end of the journey gives the taxi agent an indication that it did something wrong.
- The two points for a win at the end of a chess game tells the agent it did something right. It is up to the agent to decide which of the actions prior to the reinforcement were most responsible for it.

Supervised learning

- In supervised learning the agent observes some example input–output pairs and learns a function that maps from input to output. In component 1 above, the inputs are percepts and the output are provided by a teacher who says "Brake!" or "Turn left."
- In component 2, the inputs are camera images and the outputs again come from a teacher who says "that's a bus."
- In 3, the theory of braking is a function from states and braking actions to stopping distance in feet. In this case the output value is available directly from the agent's percepts (after the fact); the environment is the teacher.

Semi-supervised learning

- In semi-supervised learning we are given a few labeled examples and must make what we can of a large collection of unlabeled examples.
- Even the labels themselves may not be the oracular truths that we hope for. Imagine that you are trying to build a system to guess a person's age from a photo. You gather

some labeled examples by snapping pictures of people and asking their age. That's supervised learning.

- But in reality some of the people lied about their age. It's not just that there is random noise in the data; rather the inaccuracies are systematic, and to uncover them is an unsupervised learning problem involving images, self-reported ages, and true (unknown) ages. Thus, both noise and lack of labels create a continuum between supervised and unsupervised learning

Unsupervised learning: Finds patterns or clusters in unlabeled data (e.g., "good" vs. "bad" traffic days).

Reinforcement learning: Learns through rewards or punishments (e.g., no tip means poor driving).

Supervised learning: Learns from labeled input–output pairs (e.g., "Brake!" or "That's a bus").

Semi-supervised learning: Combines a few labeled examples with many unlabeled ones, often dealing with noisy or unreliable labels.

5.2 Learning Decision Trees

- Decision tree induction is one of the simplest and yet most successful forms of machine learning. We first describe the representation—the hypothesis space—and then show how to learn a good hypothesis.

5.2.1 The decision tree representation

- A decision tree represents a function that takes as input a vector of attribute values and returns a "decision"—a single output value. The input and output values can be discrete or continuous.
- For now we will concentrate on problems where the inputs have discrete values and the output has exactly two possible values; this is Boolean classification, where each example input will be classified as true (a positive example) or false (a negative example).
- Each internal node in the tree corresponds to a test of the value of one of the input attributes, A_i, and the branches from the node are labeled with the possible values of the attribute, $A_i = v_{ik}$. Each leaf node in the tree specifies a value to be returned by the function.
- As an example, we will build a decision tree to decide whether to wait for a table at a restaurant. The aim here is to learn a definition for the goal predicate WillWait. First, we list the attributes that we will consider as part of the input:

1. Alternate: whether there is a suitable alternative restaurant nearby.

2. Bar : whether the restaurant has a comfortable bar area to wait in.

3. Fri/Sat: true on Fridays and Saturdays.

4. Hungry: whether we are hungry.

5. Patrons: how many people are in the restaurant (values are None, Some, and Full).

6. Price: the restaurant's price range ($, $$, $$$).

7. Raining: whether it is raining outside.

8. Reservation: whether we made a reservation.

9. Type: the kind of restaurant (French, Italian, Thai, or burger).

10. WaitEstimate: the wait estimated by the host (0–10 minutes, 10–30, 30–60, or >60).

- Note that every variable has a small set of possible values; the value of WaitEstimate, for example, is not an integer, rather it is one of the four discrete values 0–10, 10–30, 30–60, or >60.
- For instance, an example with Patrons =Full and WaitEstimate =0–10 will be classified as positive (i.e., yes, we will wait for a table).

5.2.2 Expressiveness of decision trees

- A Boolean decision tree represents the goal as true if and only if the input attributes satisfy one of the true-valued paths in the tree.

Formally:
Goal ⇔ (Path ∨ Path ∨ …), where each path is a conjunction of attribute tests (e.g., (Patrons = Full ∧ WaitEstimate = 0–10)).

- Thus, a decision tree is equivalent to a disjunctive normal form (DNF) expression. While decision trees are concise for many problems, some functions—like the majority function—require exponentially large trees.
- There are $2^{(2^n)}$ possible Boolean functions for n attributes, meaning the hypothesis space is enormous (e.g., over 10^3 , for 20 attributes). Hence, efficient learning requires smart algorithms to find good hypotheses within this vast space.

5.2.3 Inducing decision trees from examples

- An example for a Boolean decision tree consists of an (x, y) pair, where x is a vector of values for the input attributes, and y is a single Boolean output value. A training set of 12 examples is shown in Figure 18.3. The positive examples are the ones in which the goal WillWait is true (x1, x3, . . .); the negative examples are the ones in which it is false (x2, x5, . . .).

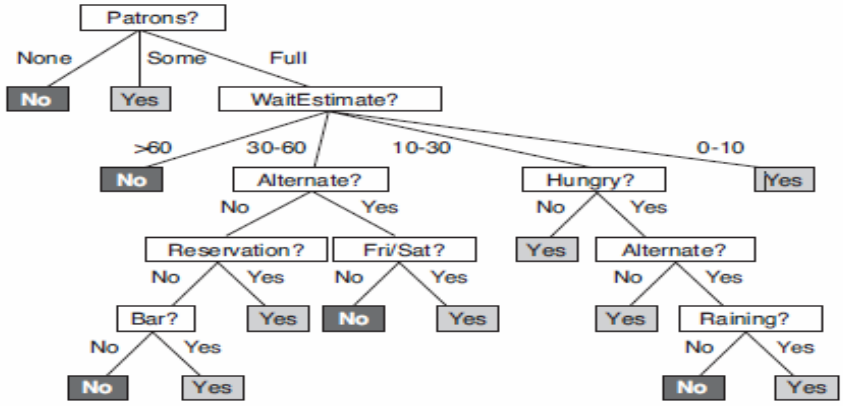


Figure 18.2 A decision tree for deciding whether to wait for a table.

Example	Input Attributes										Goal
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*WillWait*
$\mathbf{x}_1$	*Yes*	*No*	*No*	*Yes*	*Some*	*$$$*	*No*	*Yes*	*French*	*0–10*	$y_1 = Yes$
$\mathbf{x}_2$	*Yes*	*No*	*No*	*Yes*	*Full*	*$*	*No*	*No*	*Thai*	*30–60*	$y_2 = No$
$\mathbf{x}_3$	*No*	*Yes*	*No*	*No*	*Some*	*$*	*No*	*No*	*Burger*	*0–10*	$y_3 = Yes$
$\mathbf{x}_4$	*Yes*	*No*	*Yes*	*Yes*	*Full*	*$*	*Yes*	*No*	*Thai*	*10–30*	$y_4 = Yes$
$\mathbf{x}_5$	*Yes*	*No*	*Yes*	*No*	*Full*	*$$$*	*No*	*Yes*	*French*	*>60*	$y_5 = No$
$\mathbf{x}_6$	*No*	*Yes*	*No*	*Yes*	*Some*	*$$*	*Yes*	*Yes*	*Italian*	*0–10*	$y_6 = Yes$
$\mathbf{x}_7$	*No*	*Yes*	*No*	*No*	*None*	*$*	*Yes*	*No*	*Burger*	*0–10*	$y_7 = No$
$\mathbf{x}_8$	*No*	*No*	*No*	*Yes*	*Some*	*$$*	*Yes*	*Yes*	*Thai*	*0–10*	$y_8 = Yes$
$\mathbf{x}_9$	*No*	*Yes*	*Yes*	*No*	*Full*	*$*	*Yes*	*No*	*Burger*	*>60*	$y_9 = No$
$\mathbf{x}_{10}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*$$$*	*No*	*Yes*	*Italian*	*10–30*	$y_{10} = No$
$\mathbf{x}_{11}$	*No*	*No*	*No*	*No*	*None*	*$*	*No*	*No*	*Thai*	*0–10*	$y_{11} = No$
$\mathbf{x}_{12}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*$*	*No*	*No*	*Burger*	*30–60*	$y_{12} = Yes$

Figure 18.3 Examples for the restaurant domain.

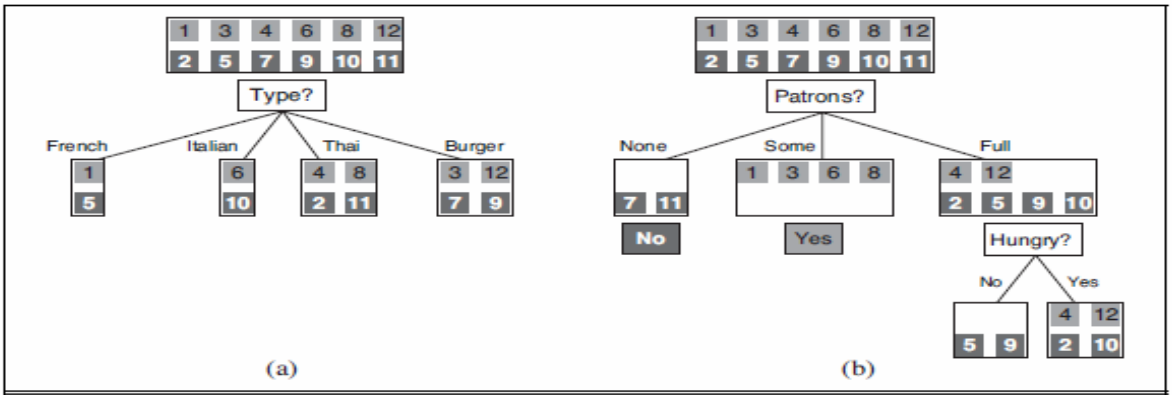


Figure 18.4 Splitting the examples by testing on attributes. At each node we show the positive (light boxes) and negative (dark boxes) examples remaining. (a) Splitting on *Type* brings us no nearer to distinguishing between positive and negative examples. (b) Splitting on *Patrons* does a good job of separating positive and negative examples. After splitting on *Patrons*, *Hungry* is a fairly good second test.

- Figure 18.4(a) shows that Type is a poor attribute, because it leaves us with four possible outcomes, each of which has the same number of positive as negative examples. On the other hand, in
- (b) we see that Patrons is a fairly important attribute, because if the value is None or Some, then we are left with example sets for which we can answer definitively (No and Yes, respectively). If the value is Full, we are left with a mixed set of examples. In general, after the first attribute test splits up the examples, each outcome is a new decision tree learning problem in itself, with fewer examples and one less attribute. There are four cases to consider for these recursive problems:

    1. If the remaining examples are all positive (or all negative), then we are done: we can answer Yes or No. Figure 18.4(b) shows examples of this happening in the None and Some branches.

    2. If there are some positive and some negative examples, then choose the best attribute to split them. Figure 18.4(b) shows Hungry being used to split the remaining examples.

    3. If there are no examples left, it means that no example has been observed for this combination of attribute values, and we return a default value calculated from the plurality classification of all the examples that were used in constructing the node's parent. These are passed along in the variable parent examples.

    4. If there are no attributes left, but both positive and negative examples, it means that these examples have exactly the same description, but different classifications. This can happen because there is an error or noise in the data; because the domain is nondeterministic; or because we can't observe an attribute that would distinguish the examples. The best we can do is return the plurality classification of the remaining examples.

```
function DECISION-TREE-LEARNING(examples, attributes, parent_examples) returns
a tree

  if examples is empty then return PLURALITY-VALUE(parent_examples)
  else if all examples have the same classification then return the classification
  else if attributes is empty then return PLURALITY-VALUE(examples)
  else
      A ← argmax_{a ∈ attributes} IMPORTANCE(a, examples)
      tree ← a new decision tree with root test A
      for each value v_k of A do
          exs ← {e : e ∈ examples and e.A = v_k}
          subtree ← DECISION-TREE-LEARNING(exs, attributes − A, examples)
          add a branch to tree with label (A = v_k) and subtree subtree
      return tree
```

Figure 18.5 The decision-tree learning algorithm. The function IMPORTANCE is described in Section 18.3.4. The function PLURALITY-VALUE selects the most common output value among a set of examples, breaking ties randomly.

The Decision-Tree-Learning algorithm builds a tree from training examples using attribute tests at internal nodes and output values at leaves. The examples themselves are not part of the tree. Although the learned tree (Figure 18.6) differs from the original (Figure 18.2), it is simpler and still consistent with all examples. The algorithm excludes unnecessary tests (like Raining and Reservation) and even uncovers new patterns (e.g., waiting for Thai food on weekends). However, it may err on unseen cases since it learns only from the provided examples.

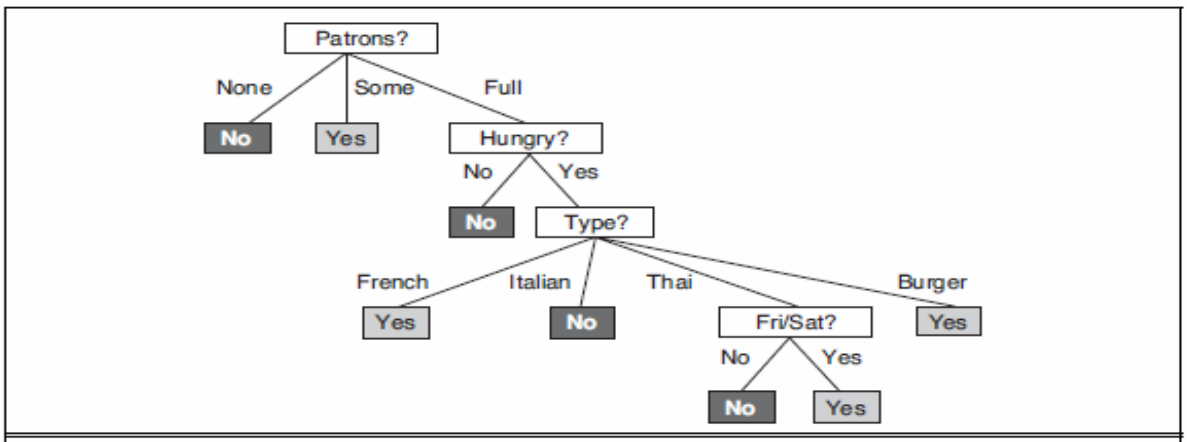


Figure 18.6 The decision tree induced from the 12-example training set.

The learned tree incorrectly predicts "not to wait" when Hungry is false, though SR would wait — more training data could fix this. However, decision trees can vary greatly with small data changes, as several attributes may have similar importance.

A learning curve (or "happy graph") shows how accuracy improves as training data increases. By testing models on different training set sizes and averaging results, we see accuracy rise toward about 95%, suggesting that more data would likely improve performance further.

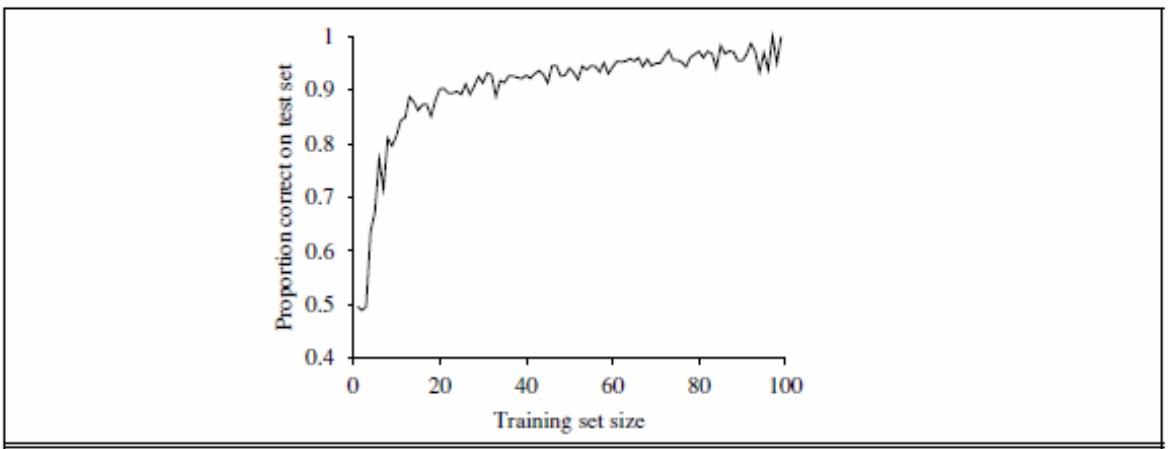


Figure 18.7 A learning curve for the decision tree learning algorithm on 100 randomly generated examples in the restaurant domain. Each data point is the average of 20 trials.

5.2.4 Choosing attribute tests

- Decision tree learning uses a greedy search to make the tree as shallow as possible by choosing the best attribute at each step. A perfect attribute completely separates positive and negative examples. A useless attribute leaves the same mix of examples as before.
- To measure how good an attribute is, we use information gain, based on entropy, which measures the uncertainty or impurity in the data.
- Entropy is a measure of the uncertainty of a random variable; acquisition of information corresponds to a reduction in entropy. A random variable with only one value—a coin that always comes up heads—has no uncertainty and thus its entropy is defined as zero; thus, we gain no information by observing its value.
- A flip of a fair coin is equally likely to come up heads or tails, 0 or 1, and we will soon show that this counts as "1 bit" of entropy. The roll of a fair four-sided die has 2 bits of entropy, because it takes two bits to describe one of four equally probable choices. Now consider an unfair coin that comes up heads 99% of the time.
- Intuitively, this coin has less uncertainty than the fair coin—if we guess heads we'll be wrong only 1% of the time—so we would like it to have an entropy measure that is close to zero, but positive. In general, the entropy of a random variable V with values v_k, each with probability $P(v_k)$, is defined as

$$\text{Entropy:} \quad H(V) = \sum_k P(v_k) \log_2 \frac{1}{P(v_k)} = -\sum_k P(v_k) \log_2 P(v_k) \ .$$

We can check that the entropy of a fair coin flip is indeed 1 bit:

$$H(Fair) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1 \ .$$

If the coin is loaded to give 99% heads, we get

$$H(Loaded) = -(0.99 \log_2 0.99 + 0.01 \log_2 0.01) \approx 0.08 \text{ bits.}$$

H(Loaded) = −(0.99 log2 0.99 + 0.01 log2 0.01) ≈ 0.08 bits.

It will help to define B(q) as the entropy of a Boolean random variable that is true with probability q:

$$B(q) = -(q \log_2 q + (1-q) \log_2 (1-q)) \ .$$

Thus, H(Loaded)=B(0.99) ≈ 0.08. Now let's get back to decision tree learning. If a training set contains p positive examples and n negative examples, then the entropy of the goal attribute on the whole set is

$$H(Goal) = B\left(\frac{p}{p+n}\right) .$$

- The restaurant training set in Figure 18.3 has p = n = 6, so the corresponding entropy is B(0.5) or exactly 1 bit. A test on a single attribute A might give us only part of this 1 bit. We can measure exactly how much by looking at the entropy remaining after the attribute test.

$$q = \frac{p}{p+n} = \frac{6}{12} = 0.5$$

$$B(0.5) = -0.5\log_2 0.5 - 0.5\log_2 0.5 = 1$$

  - ✓ Entropy = 1 bit
  - ✓ This means we are very uncertain (half Yes, half No).

- An attribute A with d distinct values divides the training set E into subsets $E_1, \ldots, E_d$. Each subset E_k has p_k positive examples and n_k negative examples, so if we go along that branch, we will need an additional $B(p_k/(p_k + n_k))$ bits of information to answer the question.
- A randomly chosen example from the training set has the kth value for the attribute with probability $(p_k + n_k)/(p + n)$, so the expected entropy remaining after testing attribute A is

$$Remainder(A) = \sum_{k=1}^{d} \frac{p_k+n_k}{p+n} B\left(\frac{p_k}{p_k+n_k}\right) .$$

- The information gain from the attribute test on A is the expected reduction in entropy:

$$Gain(A) = B\left(\frac{p}{p+n}\right) - Remainder(A) .$$

The first term $B\left(\frac{p}{p+n}\right)$ = current entropy (before the split).

The second term Remainder(A) = expected entropy after the split (weighted average of subsets).

Example with Attribute "Patrons"

The attribute Patrons has 3 possible values: None, Some, Full

From the dataset:

Here's the fully updated table with your requested changes:

Patrons	#Yes	#No	Total	Entropy
None	0	2	2	B(0) = 0
Some	4	0	4	B(1) = 0
Full	2	4	6	B(2/6) = 0.541

(a) Patrons = None

.

)pure — all No).

(b) Patrons = Some

.

)pure — all Yes).

Patrons = Full

.

Compute :

$$B\left(\tfrac{1}{3}\right) = -\tfrac{1}{3}\log_2\tfrac{1}{3} - \tfrac{2}{3}\log_2\tfrac{2}{3}.$$

$$\log_2(1/3) = \frac{\ln(1/3)}{\ln(2)} = \frac{-1.0986}{0.6931} \approx -1.5850$$

Then multiply by $-1/3$:

$$-\frac{1}{3}\log_2(1/3) = -\frac{1}{3}(-1.5850) = 0.5283$$

$$\log_2(2/3) = \frac{\ln(2/3)}{\ln(2)} = \frac{-0.4055}{0.6931} \approx -0.58496$$

Then multiply by $-2/3$:

$$-\frac{2}{3}\log_2(2/3) = -\frac{2}{3}(-0.58496) = 0.38997$$

Entropy for the (1/3, 2/3) distribution

$$B(1/3) = 0.5283 + 0.3899 = 0.9182$$

Now compute remainder:

Information Gain=1-0.4591=0.5409

So, Patrons gives us about half a bit of useful information — quite good!

The attribute Type (restaurant type) has 4 values: French, Thai, Burger, Italian.

Each has an equal mix of Yes and No (1 Yes, 1 No).

So for each: (0.5)=1

Hence: Remainder(Type)=1

Gain(Type)=1−1=0

So, "Type" tells us nothing new about whether people will wait

Attribute	Remainder(A)	Gain(A)	Conclusion
Patrons	0.459	**0.541 bits**	Very informative
Type	1.000	**0 bits**	Not informative

- Information gain measures how much an attribute reduces uncertainty.
- The higher the gain, the better the attribute.
- Since Patrons gives the maximum gain, the decision tree will split on Patrons first — making it the root node.

5.2.4 Generalization and overfitting

Overfitting: Happens when a model learns random noise instead of the true pattern.

Example: predicting if a die shows "6" using silly features (color, weight, time, etc.).

A fair die has no real pattern, so the best rule is always "No."But the decision tree might still find fake patterns (e.g., "If die = blue and fingers crossed → 6").This creates a large, meaningless tree that doesn't generalize.

Decision Tree Pruning

- After building a full tree, we remove nodes that do not really help prediction.
- A node is pruned (deleted) if:
  - ✓ The test at that node gives little or no information gain (close to 0).
  - ✓ The split looks like random noise.
- The node is replaced with a leaf node showing the majority class.

Significance Test:

- Used to check whether an attribute's information gain is statistically significant.
- Null hypothesis (H): the attribute is irrelevant → true information gain = 0.
- We compute the probability (p-value) of seeing the observed gain just by chance.
- If that probability < 5%, we say the pattern is significant (keep the split).
- Otherwise, we treat it as random and prune the node

Chi-square (χ^2) pruning

- Used to decide whether a split in a decision tree is statistically significant or just noise.
- Expected vs. actual counts
- For each subset after a split:
  -
- These are the expected numbers of positive and negative examples if the attribute were irrelevant.

Chi-square test statistic

-

- Under the **null hypothesis** (attribute irrelevant), Δ follows a χ^2 **distribution** with $d - 1$ degrees of freedom.
- If Δ exceeds the critical χ^2 value (e.g., 7.82 at 5% level for 3 df), we **reject** the null hypothesis → the split is **significant** (keep it).
- Otherwise → **prune** the node.

- Benefits of pruning:
  - ✓ Removes splits that only capture noise.
  - ✓ Makes the tree smaller, more accurate, and easier to interpret.
  - ✓ Handles noisy or mislabeled data better.

Early stopping vs. prune-after-building

- Early stopping: halt tree growth if no attribute seems good → risky.
    - Misses patterns like XOR, where no single attribute is informative alone.
- Generate-then-prune: build the full tree first, then prune using χ^2 test → better and safer.

5.2.5 Broadening the applicability of decision trees

- In order to extend decision tree induction to a wider variety of problems, a number of issues must be addressed

1. Missing Data

- Some examples have unknown attribute values.
- Problems:
    - ✓ How to classify an example when a test attribute is missing.
    - ✓ How to adjust the information gain formula for missing values.

2. Multivalued Attributes:

- Attributes with many distinct values (e.g., ExactTime) can falsely appear very informative.
- Solutions:
    - ✓ Use the Gain Ratio (information gain divided by intrinsic information).
    - ✓ Or, test one specific value at a time (e.g., A = v).

3. Continuous or Integer-Valued Attributes

- For numeric features like Height or Weight, we find a split point (e.g., Weight > 160) that gives maximum information gain.
- Done efficiently by sorting values and checking points between examples with different classes.

4. Continuous-Valued Outputs (Regression Trees)

- When predicting a numerical output (e.g., price), use a regression tree instead of a classification tree.
- Each leaf contains a linear function of input attributes, not a single class label.

5.3 Evaluating and choosing the best hypothesis

A hypothesis in machine learning is simply a candidate function or model that tries to predict the output from the input . Formally, : → where ( ) is the predicted output for a given input .

Learning, Stationarity, and Evaluation

Goal: Learn a hypothesis that predicts future data well.

Stationarity Assumption: The data distribution does not change over time.

- Each example =(,) is independent of others and drawn from the same distribution → i.i.d. (independent and identically distributed).
- This connects past data to future data — allowing learning to work.

Error Rate

- Measures how often the hypothesis ( ) gives the wrong output (( )≠).
- A low training error does not always mean good generalization to new data.

Evaluation Methods

Holdout Cross-Validation:

- Split data into training set and test set.
- Problem: wastes data (less training or less accurate testing).

k-Fold Cross-Validation:

- Split data into k equal parts.
- Train on parts, test on the remaining one.
- Repeat k times and average results.
- Common k values: 5 or 10.
- Gives better accuracy but costs more computation.

Leave-One-Out (LOOCV):

- Special case where (each example tested once).
- Very accurate but computationally expensive.

Avoiding Peeking

- **Peeking problem:** Using the test set to tune the model leaks information and gives unfair accuracy.

Solution:

- Keep test set locked away until final evaluation.
- Use a validation set (separate from the test set) to tune hyperparameters and choose the best hypothesis.

5.3.1 Model selection: Complexity versus goodness of fit

- This section explains model selection, which is the process of choosing the best model complexity (or size) to balance underfitting and overfitting.
- Example: In polynomial regression, model size = polynomial degree; in decision trees, size = number of nodes.
- Goal: Find the size that gives the lowest validation error (best generalization).

Algorithm

1. For each possible model size:
    - ✓ Train the model using a learning algorithm (e.g., decision tree learning).
    - ✓ Use cross-validation to measure training and validation error.
2. Plot error vs. model size.
    - ✓ Training error ↓ continuously.
    - ✓ Validation error forms a U-shaped curve — decreases, then increases due to overfitting.
3. Choose the model size with the lowest validation error.
4. Retrain a final model of that size using all data.
5. Test on an independent test set.

```
function CROSS-VALIDATION-WRAPPER(Learner, k, examples) returns a hypothesis
  local variables: errT, an array, indexed by size, storing training-set error rates
                   errV, an array, indexed by size, storing validation-set error rates
  for size = 1 to ∞ do
      errT[size], errV[size] ← CROSS-VALIDATION(Learner, size, k, examples)
      if errT has converged then do
          best_size ← the value of size with minimum errV[size]
          return Learner(best_size, examples)

function CROSS-VALIDATION(Learner, size, k, examples) returns two values:
          average training set error rate, average validation set error rate
  fold_errT ← 0; fold_errV ← 0
  for fold = 1 to k do
      training_set, validation_set ← PARTITION(examples, fold, k)
      h ← Learner(size, training_set)
      fold_errT ← fold_errT + ERROR-RATE(h, training_set)
      fold_errV ← fold_errV +ERROR-RATE(h, validation_set)
  return fold_errT/k, fold_errV/k
```

Figure 18.8 An algorithm to select the model that has the lowest error rate on validation data by building models of increasing complexity, and choosing the one with best empirical error rate on validation data. Here *errT* means error rate on the training data, and *errV* means error rate on the validation data. *Learner*(*size*, *examples*) returns a hypothesis whose complexity is set by the parameter *size*, and which is trained on the *examples*. PARTITION(*examples*, *fold*, *k*) splits *examples* into two subsets: a validation set of size N/k and a training set with all the other examples. The split is different for each value of *fold*.

5.3.2 From error rates to loss

- Consider the problem of classifying email messages as spam or non-spam. It is worse to classify a non-spam message as spam (and thus potentially miss an important message) than to classify spam as non-spam (and merely suffer a few seconds of annoyance).
- Therefore, a classifier with a 1% error rate, where almost all the errors are spam → non-spam, could actually be better than one with a 0.5% error rate, if most of those errors are non-spam → spam
- In machine learning, this idea is represented through a loss function.

The **loss function**, $L(x, y, \hat{y})$, is defined as the amount of utility lost when the model predicts $\hat{y} = h(x)$ instead of the correct label $y = f(x)$:

$$L(x, y, \hat{y}) = \text{Utility}(\text{result of using } y \text{ for input } x) - \text{Utility}(\text{result of using } \hat{y} \text{ for input } x)$$

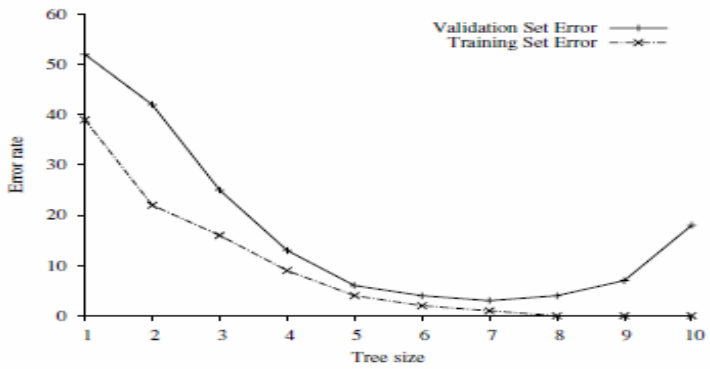


Figure 18.9 Error rates on training data (lower, dashed line) and validation data (upper, solid line) for different size decision trees. We stop when the training set error rate asymptotes, and then choose the tree with minimal error on the validation set; in this case the tree of size 7 nodes.

- Often, we use a simplified loss function, $L(y, \hat{y})$, that doesn't depend on the input .This means we can't say it's worse to misclassify a letter from Mom than from an annoying cousin, but we can still express differences in error severity.
- For example:

$$L(\text{spam}, \text{nospam}) = 1, \quad L(\text{nospam}, \text{spam}) = 10$$

- Here, misclassifying non-spam as spam is 10 times worse than the reverse.
  By definition, $L(y, y) = 0$ — there's no loss when the prediction is correct.
- For discrete outputs, each possible misclassification can be assigned a loss value.
  For real-valued outputs, small errors are better than large ones. Common loss functions include:

- $$\text{Absolute value loss (L1):} \quad L_1(y, \hat{y}) = |y - \hat{y}|$$
$$\text{Squared error loss (L2):} \quad L_2(y, \hat{y}) = (y - \hat{y})^2$$
$$\text{0/1 loss:} \quad L_{0/1}(y, \hat{y}) = \begin{cases} 0, & \text{if } y = \hat{y} \\ 1, & \text{otherwise} \end{cases}$$

- If a learning agent wants to maximize expected utility, it should instead minimize expected loss over all possible input–output pairs it will encounter. To define this expectation, we assume a probability distribution (,) over examples. Then, the expected generalization loss for a hypothesis h (with respect to a loss function L) is defined as the expected value of the loss over all examples in , the set of possible input–output pairs.

- $$GenLoss_L(h) = \sum_{(x,y)\in\mathcal{E}} L(y, h(x))\, P(x, y) ,$$
and the best hypothesis, h^*, is the one with the minimum expected generalization loss:
$$h^* = \underset{h\in\mathcal{H}}{\operatorname{argmin}}\, GenLoss_L(h) .$$
Because $P(x, y)$ is not known, the learning agent can only *estimate* generalization loss with **empirical loss** on a set of examples, E:
$$EmpLoss_{L,E}(h) = \frac{1}{N} \sum_{(x,y)\in E} L(y, h(x)) .$$
The estimated best hypothesis $\hat{h}^*$ is then the one with minimum empirical loss:
$$\hat{h}^* = \underset{h\in\mathcal{H}}{\operatorname{argmin}}\, EmpLoss_{L,E}(h) .$$

  There are four main reasons why the learned hypothesis $\hat{h}^*$ may differ from the true function :

  1. Unrealiability – The true function may not exist within the hypothesis space , or other hypotheses may be preferred due to the learning algorithm's design.

  2. Variance – Different training sets (even from the same function) can produce different hypotheses, leading to variation in predictions. Variance causes error but decreases as the number of training examples increases.

  3. Noise – The target function may be nondeterministic or affected by unobserved factors, causing unpredictable differences in output values for the same input.

  4. Computational Complexity – When H is large or complex, it may be computationally infeasible to find the optimal hypothesis; instead, we rely on approximate or local search methods.

Together, these sources lead to an approximation of an estimation of the true function f.

- In **small-scale learning** (few examples), generalization error mainly comes from approximation and estimation errors.
- In **large-scale learning** (millions of examples), the main limitation is computational complexity—we have enough data and expressive models, but cannot fully optimize them.

5.3.3 Regularization

- An alternative to cross-validation for model selection is to directly minimize the total cost of a hypothesis, defined as:

$$Cost(h) = EmpLoss(h) + \lambda\, Complexity(h)$$
$$\hat{h}^* = \operatorname*{argmin}_{h \in \mathcal{H}} Cost(h)\ .$$

- Here, λ is a positive parameter that balances the trade-off between empirical loss (training error) and model complexity.
- This process is called regularization, which explicitly penalizes complex hypotheses to prefer simpler, more generalizable models. The regularization function depends on the hypothesis space — for example, for polynomials, it can be the sum of squares of coefficients, which discourages highly wiggly curves.
- Another way to simplify models is through feature selection, which removes irrelevant attributes (e.g., χ^2 pruning).
- In some formulations, empirical loss and complexity can be measured on the same scale—in bits.This leads to the Minimum Description Length (MDL) principle, which selects the hypothesis requiring the fewest bits to encode both the model and the data it explains.
- While MDL works well theoretically, it can be difficult to apply in small problems because the encoding scheme (e.g., how to represent a decision tree) affects the results.

Feature	**Generalization Loss**	**Empirical Loss**
Definition	Error on unseen data	Error on training data
Also called	Expected loss / True risk	Training loss
Goal	Minimize error on new data	Minimize error on known data
Behavior	Can increase if model overfits	Usually decreases with model complexity

5.4 The theory of learning

- Any learning algorithm that returns hypotheses that are probably approximately correct is called a PAC (Probability Approximately Correct) learning algorithm; we can use this approach to provide bounds on the performance of various learning algorithms
- PAC-learning theorems, like all theorems, are logical consequences of axioms. When a theorem (as opposed to, say, a political pundit) states something about the future based on the past, the axioms have to provide the "juice" to make that connection.
- The simplest PAC theorems deal with Boolean functions, for which the 0/1 loss is appropriate. The error rate of a hypothesis h, defined informally earlier, is defined formally here as the expected generalization error for examples drawn from the stationary distribution

$$\text{error}(h) = GenLoss_{L_{0/1}}(h) = \sum_{x,y} L_{0/1}(y, h(x))\, P(x, y) \ .$$

- In other words, error(h) is the probability that h misclassifies a new example. This is the same quantity being measured experimentally by the learning curves shown earlier.
- In PAC learning, we aim to show that with enough training examples, any hypothesis consistent with the data is likely to be approximately correct.
- Key Definitions:
  - ✓ Approximately correct hypothesis: A hypothesis such that error( )≤ , where is a small constant.
  - ✓ -ball: The set of hypotheses within error of the true function .
  - ✓ H_{bad}: The set of hypotheses outside the -ball, i.e., those with error > .
- Bounding the Probability of a Bad Hypothesis Being Consistent:
- Let ∈ $_{bad}$ be a hypothesis with error( )> . Then: The probability that agrees with a single randomly drawn example is at most 1− .Since examples are drawn independently, the probability that agrees with all examples is at most: (1−)
- This is an exponentially decreasing bound. As increases, the probability that any bad hypothesis remains consistent with all examples drops rapidly.
- Ensure that with high probability (≥ 1−), any hypothesis consistent with the training data has error ≤ . This is the essence of Probably Approximately Correct (PAC) learning.

Bounding Bad Hypotheses

- Let $_{bad}$ be the set of hypotheses with error > .
- The probability that a bad hypothesis is consistent with examples is:

(is consistent)≤(1−)

- Using the union bound:

($_{bad}$ contains a consistent hypothesis)≤| $_{bad}$|(1−) ≤| |(1−)

Sample Complexity Bound

To ensure this probability is ≤ , we solve:

Using the inequality ,we get:

Given that $1 - \epsilon \leq e^{-\epsilon}$, we can achieve this if we allow the algorithm to see

$$N \geq \frac{1}{\epsilon}\left(\ln\frac{1}{\delta} + \ln|\mathcal{H}|\right)$$

5.4.1 PAC learning example: Learning decision lists

- A decision list consists of a series of tests, each of which is a conjunction of literals.
- If a test succeeds when applied to an example description, the decision list specifies the value to be returned.
- If the test fails, processing continues with the next test in the list. Decision lists resemble decision trees, but their overall structure is simpler: they branch only in one direction. In contrast, the individual tests are more complex

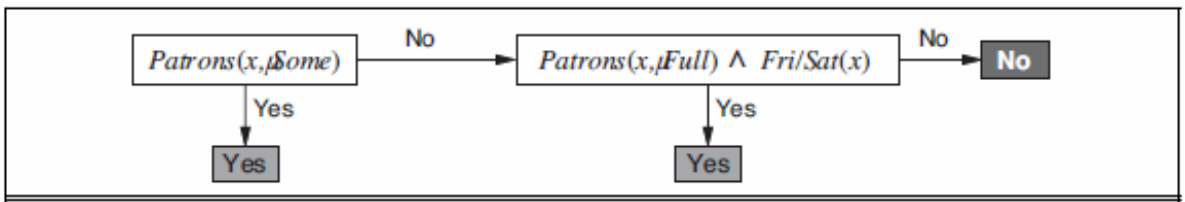


Figure 18.10 A decision list for the restaurant problem.

18.10 shows a decision list that represents the following hypothesis:

$$WillWait \Leftrightarrow (Patrons = Some) \vee (Patrons = Full \wedge Fri/Sat) .$$

- If we allow tests of arbitrary size, then decision lists can represent any Boolean function
  On the other hand, if we restrict the size of each test to at most k literals, then it is possible for the learning algorithm to generalize successfully from a small number of examples. We call this language k-DL
- k-DL includes as a subset the language k-DT, the set of all decision trees of depth at most k. It is important to remember that the particular language referred to by k-DL depends on the attributes used to describe the examples. We will use the notation k-DL(n) to denote a k-DL language using n Boolean attributes
- The first task is to show that k-DL is learnable—that is, that any function in k-DL can be approximated accurately after training on a reasonable number of examples. To do this, we need to calculate the number of hypotheses in the language. Let the language of tests— conjunctions of at most k literals using n attributes—be Conj (n, k). Because a decision list is constructed of tests, and because each test can be attached to either a Yes or a No outcome or can be absent from the decision list, there are at most $3^{|\text{Conj (n,k)}|}$ distinct sets of component tests. Each of these sets of tests can be in any order, so

$$|k\text{-DL}(n)| \leq 3^{|Conj(n,k)|}\,|Conj(n,k)|!$$

- The number of conjunctions of k literals from n attributes is given by

$$|Conj(n,k)| = \sum_{i=0}^{k} \binom{2n}{i} = O(n^k) .$$

- Hence, after some work, we obtain

$$|k\text{-DL}(n)| = 2^{O(n^k \log_2(n^k))} .$$

We can plug this into Equation (18.1) to show that the number of examples needed for PAC-learning a k-DL function is polynomial in n:

$$N \geq \frac{1}{\epsilon}\left(\ln\frac{1}{\delta} + O(n^k \log_2(n^k))\right) .$$

Therefore, any algorithm that returns a consistent decision list will PAC-learn a k-DL function in a reasonable number of examples, for small k.

function DECISION-LIST-LEARNING(*examples*) **returns** a decision list, or *failure*

if *examples* is empty **then return** the trivial decision list *No*
$t \leftarrow$ a test that matches a nonempty subset $examples_t$ of *examples*
such that the members of $examples_t$ are all positive or all negative
if there is no such t **then return** *failure*
if the examples in $examples_t$ are positive **then** $o \leftarrow Yes$ **else** $o \leftarrow No$
return a decision list with initial test t and outcome o and remaining tests given by
DECISION-LIST-LEARNING(*examples* $-$ $examples_t$)

Figure 18.11 An algorithm for learning decision lists.

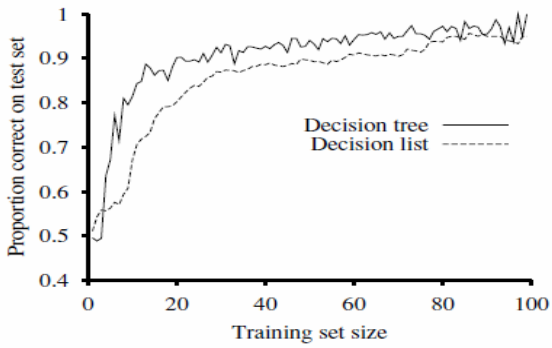


Figure 18.12 Learning curve for DECISION-LIST-LEARNING algorithm on the restaurant data. The curve for DECISION-TREE-LEARNING is shown for comparison.

- The next task is to find an efficient algorithm that returns a consistent decision list. We will use a greedy algorithm called DECISION-LIST-LEARNING that repeatedly finds a test that agrees exactly with some subset of the training set.
- Once it finds such a test, it adds it to the decision list under construction and removes the corresponding examples. It then constructs the remainder of the decision list, using just the remaining examples. This is repeated until there are no examples left.

5.4 Regression and classification with linear models

- Now we move from decision trees and lists to a new hypothesis space — linear functions. These have been used for centuries to model continuous-valued inputs. We begin with the simplest form: univariate linear regression (fitting a straight line). Later sections cover multivariate regression and how linear functions can be converted into classifiers using hard or soft thresholds.

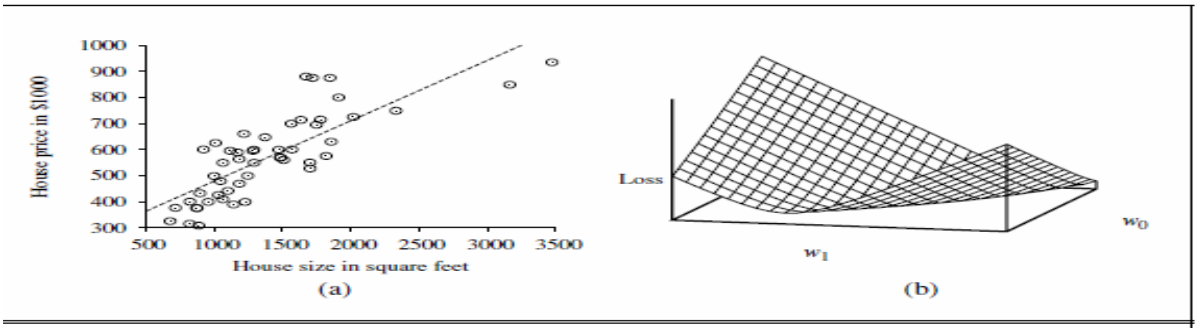


Figure 18.13 (a) Data points of price versus floor space of houses for sale in Berkeley, CA, in July 2009, along with the linear function hypothesis that minimizes squared error loss: $y = 0.232x + 246$. (b) Plot of the loss function $\sum_j (w_1 x_j + w_0 - y_j)^2$ for various values of w_0, w_1. Note that the loss function is convex, with a single global minimum.

5.4.1 Univariate linear regression

- A univariate linear function (a straight line) with input x and output y has the form

$y = w_1 x + w_0$, where w_0 and w_1 are real-valued coefficients to be learned. We use the letter w because we think of the coefficients as weights; the value of y is changed by changing the relative weight of one term or another.

We'll define w to be the vector $[w_0, w_1]$, and define

$$h_w(x) = w_1 x + w_0$$

Figure 18.13(a) shows a training set of points where each represents a house's size (x) and price (y). The goal of linear regression is to find the best-fitting line by determining weights [w , w] that minimize the empirical loss. Traditionally, this uses the squared loss function (L)—the sum of squared differences between predicted and actual values, a method dating back to Gauss.

$$Loss(h_{\mathbf{w}}) = \sum_{j=1}^{N} L_2(y_j, h_{\mathbf{w}}(x_j)) = \sum_{j=1}^{N} (y_j - h_{\mathbf{w}}(x_j))^2 = \sum_{j=1}^{N} (y_j - (w_1 x_j + w_0))^2 .$$

We would like to find $\mathbf{w}^* = \text{argmin}_{\mathbf{w}} Loss(h_{\mathbf{w}})$. The sum $\sum_{j=1}^{N} (y_j - (w_1 x_j + w_0))^2$ is minimized when its partial derivatives with respect to w_0 and w_1 are zero:

$$\frac{\partial}{\partial w_0} \sum_{j=1}^{N} (y_j - (w_1 x_j + w_0))^2 = 0 \text{ and } \frac{\partial}{\partial w_1} \sum_{j=1}^{N} (y_j - (w_1 x_j + w_0))^2 = 0 . \quad (18.2)$$

These equations have a unique solution:

$$w_1 = \frac{N(\sum x_j y_j) - (\sum x_j)(\sum y_j)}{N(\sum x_j^2) - (\sum x_j)^2}; \quad w_0 = (\sum y_j - w_1(\sum x_j))/N . \quad (18.3)$$

- In Figure 18.13(a), the best-fit line has weights w = 0.232 and w = 246.
  The weight space for linear regression (defined by w and w) is two-dimensional, and the loss surface is convex—meaning it has a single global minimum and no local minima.

- To find these optimal weights, we minimize the L loss. While linear models have a closed-form solution, more complex models often do not. In such cases, we use gradient

descent, an optimization method that starts from any point in weight space and repeatedly moves downhill along the gradient until the loss reaches its minimum.

$$\mathbf{w} \leftarrow \text{any point in the parameter space}$$
$$\textbf{loop until convergence do}$$
$$\quad \textbf{for each } w_i \textbf{ in w do}$$
$$\quad\quad w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} Loss(\mathbf{w})$$

- The parameter α, called the learning rate, controls how big a step we take when minimizing the loss. It can be constant or decrease over time. In univariate regression, the loss function is quadratic, so its partial derivatives (slopes) are linear.
- Using basic calculus rules — $\partial/\partial x\ (x^2) = 2x$ and $\partial/\partial x\ (x) = 1$ — we can find these derivatives for a single training example (x, y) to determine how the weights should be adjusted during learning.

$$\begin{aligned}\frac{\partial}{\partial w_i} Loss(\mathbf{w}) &= \frac{\partial}{\partial w_i}(y - h_{\mathbf{w}}(x))^2 \\ &= 2(y - h_{\mathbf{w}}(x)) \times \frac{\partial}{\partial w_i}(y - h_{\mathbf{w}}(x)) \\ &= 2(y - h_{\mathbf{w}}(x)) \times \frac{\partial}{\partial w_i}(y - (w_1 x + w_0))\ , \end{aligned} \tag{18.5}$$

applying this to both w_0 and w_1 we get:

$$\frac{\partial}{\partial w_0} Loss(\mathbf{w}) = -2(y - h_{\mathbf{w}}(x))\ ; \qquad \frac{\partial}{\partial w_1} Loss(\mathbf{w}) = -2(y - h_{\mathbf{w}}(x)) \times x$$

Then, plugging this back into Equation (18.4), and folding the 2 into the unspecified learning rate α, we get the following learning rule for the weights:

$$w_0 \leftarrow w_0 + \alpha\,(y - h_{\mathbf{w}}(x))\ ; \quad w_1 \leftarrow w_1 + \alpha\,(y - h_{\mathbf{w}}(x)) \times x$$

These updates make intuitive sense: if $h_{\mathbf{w}}(x) > y$, i.e., the output of the hypothesis is too large, reduce w_0 a bit, and reduce w_1 if x was a positive input but increase w_1 if x was a negative input.

The preceding equations cover one training example. For N training examples, we want to minimize the sum of the individual losses for each example. The derivative of a sum is the sum of the derivatives, so we have:

$$w_0 \leftarrow w_0 + \alpha \sum (y_j - h_{\mathbf{w}}(x_j))\ ; \quad w_1 \leftarrow w_1 + \alpha \sum (y_j - h_{\mathbf{w}}(x_j)) \times x_j\ .$$

- Batch gradient descent updates weights after processing all training examples, guaranteeing convergence to the global minimum if the learning rate (α) is small enough, though it can be slow.
- Batch gradient descent updates weights after processing all training examples, guaranteeing convergence to the global minimum if the learning rate (α) is small enough, though it can be slow.
- Stochastic gradient descent (SGD) updates weights after each individual training example, making it faster and suitable for both online and offline learning. However,

with a fixed α, it may oscillate around the minimum; using a decreasing learning rate helps ensure convergence.

5.4.2 Multivariate linear regression

- We can easily extend to multivariate linear regression problems, in which each example x_j is an n-element vector. Our hypothesis space is the set of functions of the form

$$h_{sw}(\mathbf{x}_j) = w_0 + w_1 x_{j,1} + \cdots + w_n x_{j,n} = w_0 + \sum_i w_i x_{j,i} .$$

- The w_0 term, the intercept, stands out as different from the others. We can fix that by inventing a dummy input attribute, $x_{j,0}$, which is defined as always equal to 1. Then h is simply the dot product of the weights and the input vector (or equivalently, the matrix product of the transpose of the weights and the input vector):

$$h_{sw}(\mathbf{x}_j) = \mathbf{w} \cdot \mathbf{x}_j = \mathbf{w}^\top \mathbf{x}_j = \sum_i w_i x_{j,i} .$$

The best vector of weights, $\mathbf{w}^*$, minimizes squared-error loss over the examples:

$$\mathbf{w}^* = \operatorname*{argmin}_{\mathbf{w}} \sum_j L_2(y_j, \mathbf{w} \cdot \mathbf{x}_j) .$$

Multivariate linear regression is actually not much more complicated than the univariate case we just covered. Gradient descent will reach the (unique) minimum of the loss function; the update equation for each weight w_i is

$$w_i \leftarrow w_i + \alpha \sum_j x_{j,i}(y_j - h_{\mathbf{w}}(\mathbf{x}_j)) . \qquad (18.6)$$

It is also possible to solve analytically for the w that minimizes loss. Let y be the vector of outputs for the training examples, and **X** be the **data matrix**, i.e., the matrix of inputs with one n-dimensional example per row. Then the solution

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

minimizes the squared error.

- In univariate linear regression, overfitting isn't a concern. But in multivariate regression, irrelevant features can cause overfitting. To prevent this, regularization is used—it minimizes both the empirical loss and the model's complexity, ensuring the model fits the data well without becoming overly complex.

$$Cost(h) = EmpLoss(h) + \lambda\, Complexity(h) .$$

For linear functions the complexity can be specified as a function of the weights. We can consider a family of regularization functions:

$$Complexity(h_{\mathbf{w}}) = L_q(\mathbf{w}) = \sum_i |w_i|^q .$$

- With q = 1, we get L1 regularization, which minimizes the sum of absolute weight values; with q = 2, L2 regularization minimizes the sum of squared weights. L1 regularization often produces a sparse model, setting many weights to zero and ignoring irrelevant features—similar to decision trees. Such models are simpler, easier to interpret, and less prone to overfitting.

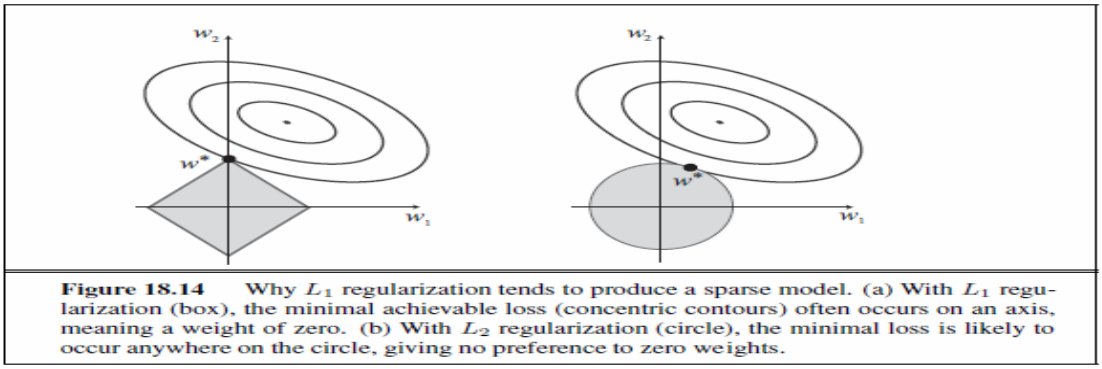


Figure 18.14 Why L_1 regularization tends to produce a sparse model. (a) With L_1 regularization (box), the minimal achievable loss (concentric contours) often occurs on an axis, meaning a weight of zero. (b) With L_2 regularization (circle), the minimal loss is likely to occur anywhere on the circle, giving no preference to zero weights.

- L_1 regularization creates a diamond-shaped constraint in weight space, while L_2 regularization forms a circular one. In L_1, the sharp corners often touch the loss contours at points where some weights are zero, leading to sparse models. In contrast, L_2's circular shape rarely hits the axes, so it keeps all weights nonzero.

5.4.3 Linear classifiers with a hard threshold

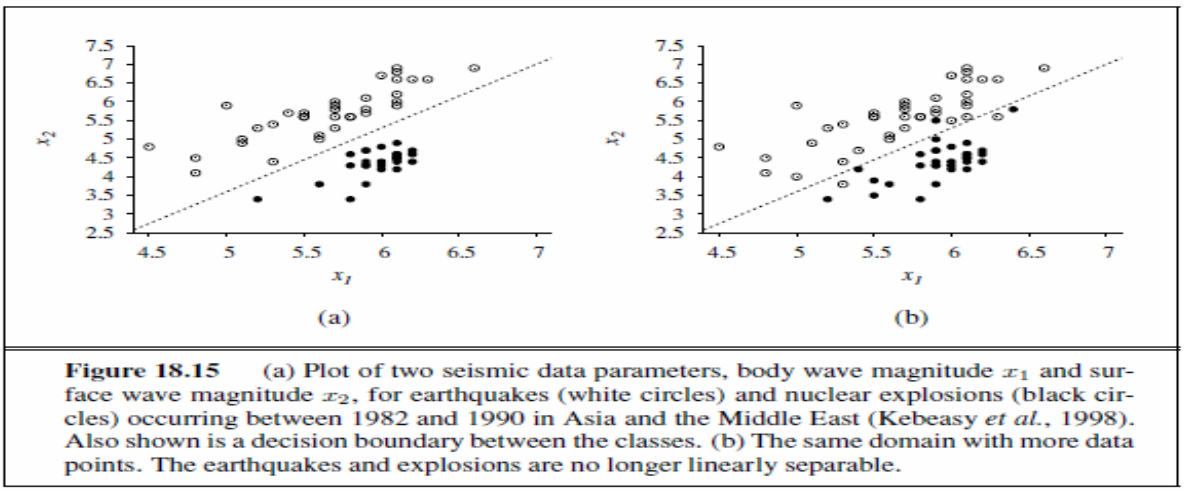


Figure 18.15 (a) Plot of two seismic data parameters, body wave magnitude x_1 and surface wave magnitude x_2, for earthquakes (white circles) and nuclear explosions (black circles) occurring between 1982 and 1990 in Asia and the Middle East (Kebeasy *et al.*, 1998). Also shown is a decision boundary between the classes. (b) The same domain with more data points. The earthquakes and explosions are no longer linearly separable.

Linear functions can also perform classification. In Figure 18.15(a), data points represent two classes — earthquakes and underground explosions — each described by two features: x (body wave magnitude) and x (surface wave magnitude).The goal is to learn a hypothesis h(x , x) that classifies new points correctly, outputting 0 for earthquakes and 1 for explosions.

- Decision boundary separates two classes; if it's a straight line, it's a linear separator, and the data are linearly separable. For two features, the separator can be written as:

- Using a dummy input ,the perceptron hypothesis is:

- The perceptron learning rule updates weights for a single example as:

- Correct prediction → no change
- , →increase weights for positive inputs
- , →decrease weights for positive inputs

Applied one example at a time, it converges to a perfect separator if data are linearly separable. For non-separable or noisy data, fixed learning rates may fail, but using a decaying learning rate ( )=1000/(1000+)α(t)=1000/(1000+t) improves convergence to a minimum-error solution, though perfect separation may still be impossible.

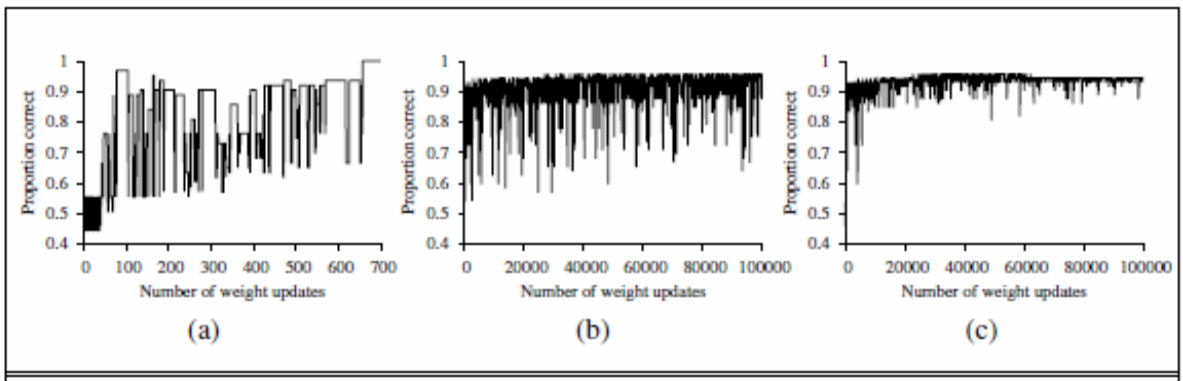


Figure 18.16 (a) Plot of total training-set accuracy vs. number of iterations through the training set for the perceptron learning rule, given the earthquake/explosion data in Figure 18.15(a). (b) The same plot for the noisy, non-separable data in Figure 18.15(b); note the change in scale of the x-axis. (c) The same plot as in (b), with a learning rate schedule $\alpha(t) = 1000/(1000 + t)$.

5.4.4 Linear classification with logistic regression

- Passing a linear function through a hard threshold creates a linear classifier, but it has drawbacks: The hypothesis ( ) is discontinuous and non-differentiable, making learning with the perceptron unpredictable. Predictions are all-or-nothing (0 or 1), even for points near the decision boundary, which can be unrealistic.
- These issues are addressed by softening the threshold using a continuous, differentiable function. Examples include:
  - ✓ The probit function (integral of the standard normal distribution)
  - ✓ The logistic function:

$$\textbf{Logistic}(z) = \frac{1}{1 + e^{-z}}$$

This allows smooth, probabilistic predictions and enables gradient-based learning.

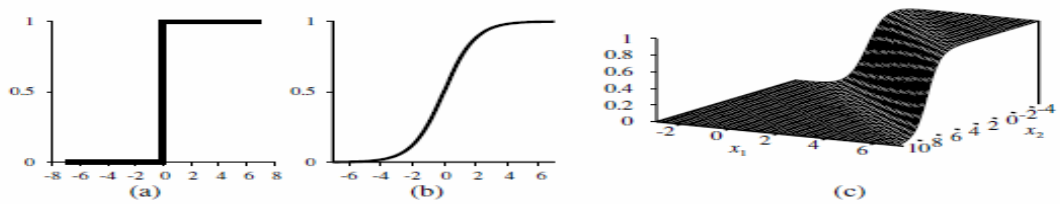


Figure 18.17 (a) The hard threshold function $Threshold(z)$ with 0/1 output. Note that the function is nondifferentiable at $z = 0$. (b) The logistic function, $Logistic(z) = \frac{1}{1+e^{-z}}$, also known as the sigmoid function. (c) Plot of a logistic regression hypothesis $h_{\mathbf{w}}(\mathbf{x}) = Logistic(\mathbf{w} \cdot \mathbf{x})$ for the data shown in Figure 18.15(b).

$$h_{\mathbf{w}}(\mathbf{x}) = Logistic(\mathbf{w} \cdot \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w} \cdot \mathbf{x}}} .$$

- In the earthquake/explosion example, a logistic hypothesis outputs a value between 0 and 1, interpretable as the probability of class 1. The decision boundary becomes soft: inputs at the center yield 0.5, while points farther away approach 0 or 1.
- Fitting the weights of this model is called logistic regression. There's no closed-form solution, but gradient descent can be applied. Using the L_2 loss and letting denote the logistic function (with derivative ′), the gradient for a single example (,) is computed using the chain rule:

$$\frac{\partial g(f(x))}{\partial x} = g'(f(x)) \cdot \frac{\partial f(x)}{\partial x}$$

- This allows straightforward optimization of the weights.

$$\begin{aligned}
\frac{\partial}{\partial w_i} Loss(\mathbf{w}) &= \frac{\partial}{\partial w_i} (y - h_{\mathbf{w}}(\mathbf{x}))^2 \\
&= 2(y - h_{\mathbf{w}}(\mathbf{x})) \times \frac{\partial}{\partial w_i} (y - h_{\mathbf{w}}(\mathbf{x})) \\
&= -2(y - h_{\mathbf{w}}(\mathbf{x})) \times g'(\mathbf{w} \cdot \mathbf{x}) \times \frac{\partial}{\partial w_i} \mathbf{w} \cdot \mathbf{x} \\
&= -2(y - h_{\mathbf{w}}(\mathbf{x})) \times g'(\mathbf{w} \cdot \mathbf{x}) \times x_i .
\end{aligned}$$

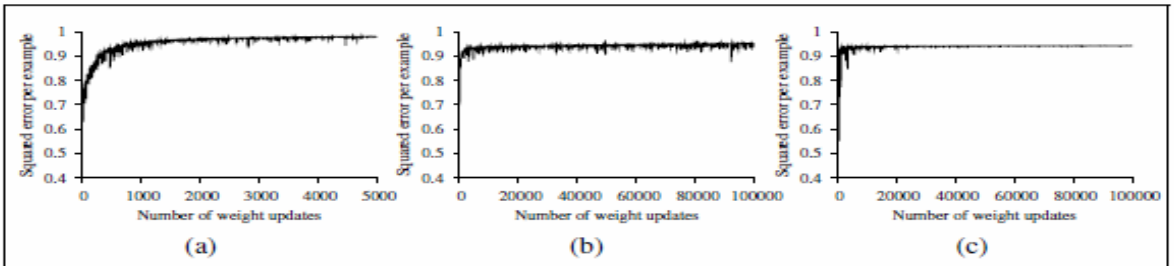


Figure 18.18 Repeat of the experiments in Figure 18.16 using logistic regression and squared error. The plot in (a) covers 5000 iterations rather than 1000, while (b) and (c) use the same scale.

The derivative g' of the logistic function satisfies $g'(z) = g(z)(1 - g(z))$, so we have

$$g'(\mathbf{w} \cdot \mathbf{x}) = g(\mathbf{w} \cdot \mathbf{x})(1 - g(\mathbf{w} \cdot \mathbf{x})) = h_{\mathbf{w}}(\mathbf{x})(1 - h_{\mathbf{w}}(\mathbf{x}))$$

so the weight update for minimizing the loss is

$$w_i \leftarrow w_i + \alpha\,(y - h_{\mathbf{w}}(\mathbf{x})) \times h_{\mathbf{w}}(\mathbf{x})(1 - h_{\mathbf{w}}(\mathbf{x})) \times x_i\ .$$

- When the logistic regression model is used instead of a hard-threshold classifier, it converges more smoothly and predictably.
  - ❑ In linearly separable data, it converges more slowly but steadily.
  - ❑ In noisy or nonseparable data, it converges faster and more reliably.
- Because of its stability and probabilistic outputs, logistic regression is widely used in real-world fields such as medicine, marketing, credit scoring, and public health.

5.5 Explanation-based learning

- The technique of memoization has long been used in computer science to speed up programs by saving the results of computation.
- The basic idea of memo functions is to accumulate a database of input–output pairs; when the function is called, it first checks the database to see whether it can avoid solving the problem from scratch. Explanation-based learning takes this a good deal further, by creating general rules that cover an entire class of cases.
- In the case of differentiation, memoization would remember that the derivative of X^2 with respect to X is 2X, but would leave the agent to calculate the derivative of Z^2 with respect to Z from scratch. We would like to be able to extract the general rule that for any arithmetic unknown u, the derivative of u^2 with respect to u is 2u. (An even more general rule for un can also be produced, but the current example suffices to make the point.) In logical terms, this is expressed by the rule

$$ArithmeticUnknown(u) \Rightarrow Derivative(u^2, u) = 2u$$

- Explanation-based learning (EBL) allows a system to generalize from a specific example to a broader rule. Once a rule is understood—like the derivative of being —any new instance matching the rule can be solved immediately.
- EBL turns learned steps into reusable building blocks, making complex problem-solving more efficient. Understanding often follows examples, since explaining why something works is easier than inventing it from scratch.

5.5.1 Extracting general rules from examples

- The essence of EBL can be summarized as follows:
- Construct an explanation: Use prior knowledge to explain why the observation (or solution) is correct. This explanation could be a logical proof or any well-defined reasoning process.
- Generalize the explanation: Identify the necessary conditions that allowed the solution steps to succeed.
- Form a general rule: Define a class of cases for which the same explanation applies, creating a rule that can solve all such cases efficiently.

Example setup:

- Instead of the complex derivative problem, consider simplifying $1 \times (0 + X)$. The knowledge base includes rules such as:
  - ✓ $0 + Y = Y$
  - ✓ $1 \times Y = Y$
- Using EBL, the system can reason through the simplification once and then generalize a rule like:
  - ✓ $1 \times (0 + U) = U$
- This rule can now be applied immediately to any similar expression, without recomputing step by step.

$$
\begin{aligned}
&Rewrite(u, v) \wedge Simplify(v, w) \Rightarrow Simplify(u, w) \\
&Primitive(u) \Rightarrow Simplify(u, u) . \\
&ArithmeticUnknown(u) \Rightarrow Primitive(u) . \\
&Number(u) \Rightarrow Primitive(u) . \\
&Rewrite(1 \times u, u) . \\
&Rewrite(0 + u, u) . \\
&\quad \vdots
\end{aligned}
$$

- EBL constructs two proof trees simultaneously: A specific proof for the given example. A generalized (variabilized) proof where constants are replaced by variables, following the same inference steps.

- During generalization, some variables get instantiated to match rules (e.g., x = 1, y = 0). From the generalized proof tree, a rule is formed using the leaves as conditions:

  Rewrite(1 × (0 + z),0 + z) ∧ Rewrite(0 + z, z) ∧ ArithmeticUnknown(z)⇒

  Simplify(1 × (0 + z), z)

- Notice that the first two conditions on the left-hand side are true regardless of the value of z.

  We can therefore drop them from the rule, yielding

  ArithmeticUnknown(z) ⇒ Simplify(1 × (0 + z), z) .

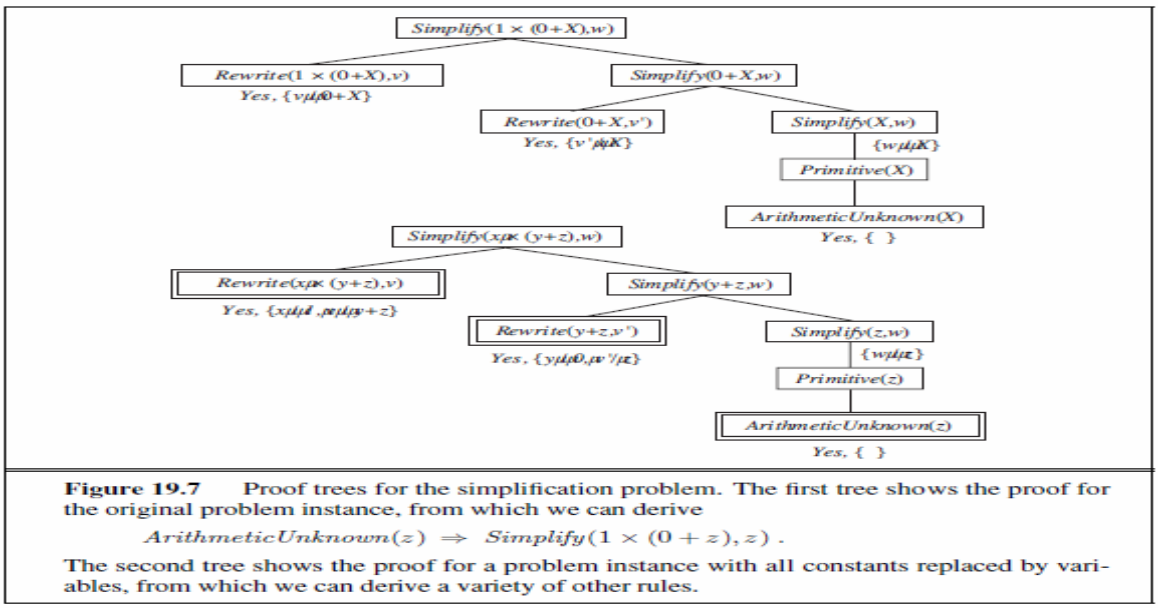


Figure 19.7 Proof trees for the simplification problem. The first tree shows the proof for the original problem instance, from which we can derive

$$ArithmeticUnknown(z) \Rightarrow Simplify(1 \times (0 + z), z) .$$

The second tree shows the proof for a problem instance with all constants replaced by variables, from which we can derive a variety of other rules.

The basic EBL process works as follows:

1. Given an example, construct a proof that the goal predicate applies to the example using the available background knowledge.

2. In parallel, construct a generalized proof tree for the variabilized goal using the same inference steps as in the original proof.

3. Construct a new rule whose left-hand side consists of the leaves of the proof tree and whose right-hand side is the variabilized goal (after applying the necessary bindings from the generalized proof).

4. Drop any conditions from the left-hand side that are true regardless of the values of the variables in the goal.

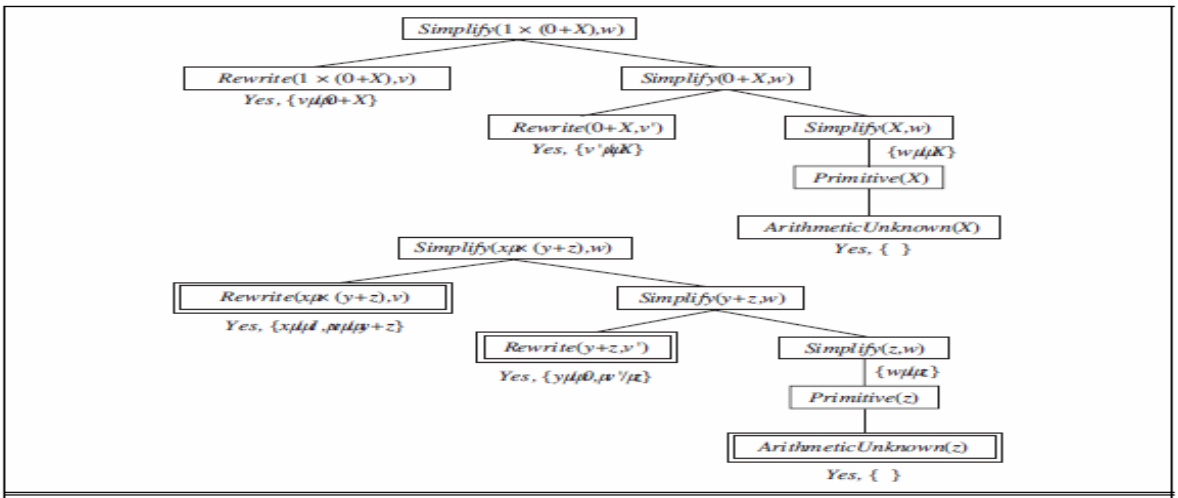


Figure 19.7 Proof trees for the simplification problem. The first tree shows the proof for the original problem instance, from which we can derive

$$ArithmeticUnknown(z) \Rightarrow Simplify(1 \times (0 + z), z) .$$

The second tree shows the proof for a problem instance with all constants replaced by variables, from which we can derive a variety of other rules.

5.5.2 Improving efficiency

EBL can generate multiple generalized rules from a single proof tree, depending on where the tree is pruned. Examples:

Primitive(z) ⇒ Simplify(1 × (0 + z), z) (covers numbers)

Simplify(y + z, w) ⇒ Simplify(1 × (y + z), w) (more general)

Choosing which rule to add depends on efficiency, considering:

1. Reasoning slowdown: Too many rules can increase search branching.

2. Speed gains: Rules should significantly reduce proof steps and avoid dead ends.

3. Generality: Rules should cover as many cases as possible.

Operationality helps balance this:

- A subgoal is operational if it is easy to solve (e.g., Primitive(z) vs. Simplify(y + z,w)).
- Prune branches when an operational subgoal is reached to create efficient rules.
- Finally, the cost of solving a given subgoal depends on what other rules are available in the knowledge base. It can go up or down as more rules are added. Thus, EBL systems really face a very complex optimization problem in trying to maximize the efficiency of a given initial knowledge base.
- It is sometimes possible to derive a mathematical model of the effect on overall efficiency of adding a given rule and to use this model to select the best rule to add. The analysis can become very complicated, however, especially when recursive rules are involved.

- One promising approach is to address the problem of efficiency empirically, simply by adding several rules and seeing which ones are useful and actually speed things up.
- Properly applied EBL can yield significant speedups in complex systems, such as real-time natural language translation

5.6 Statistical Learning

- Here, the data are evidence—that is, instantiations of some or all of the random variables describing the domain.
- Consider a simple example. Our favorite Surprise candy comes in two flavors: cherry(yum) and lime (ugh). The manufacturer has a peculiar sense of humor and wraps each piece of candy in the same opaque wrapper, regardless of flavor.
- The candy is sold in very large bags, of which there are known to be five kinds—again, indistinguishable from the outside:

h_1: 100% cherry,
h_2: 75% cherry + 25% lime,
h_3: 50% cherry + 50% lime,
h_4: 25% cherry + 75% lime,
h_5: 100% lime .

- Given a new bag of candy, the random variable H (for hypothesis) denotes the type of the bag, with possible values h_1 through h_5. H is not directly observable, of course. As the pieces of candy are opened and inspected, data are revealed—D_1, D_2, . . ., D_N, where each D_i is a random variable with possible values cherry and lime.
- The basic task faced by the agent is to predict the flavor of the next piece of candy. Despite its apparent triviality, this scenario serves to introduce many of the major issues. The agent really does need to infer a theory of its world, albeit a very simple one.
- Bayesian learning simply calculates the probability of each hypothesis, given the data, and makes predictions on that basis. That is, the predictions are made by using all the hypotheses, weighted by their probabilities, rather than by using just a single "best" hypothesis.
- In this way, learning is reduced to probabilistic inference. Let D represent all the data, with observed value d; then the probability of each hypothesis is obtained by Bayes' rule:

$$P(h_i \mid \mathbf{d}) = \alpha P(\mathbf{d} \mid h_i) P(h_i) \,. \tag{20.1}$$

Now, suppose we want to make a prediction about an unknown quantity X. Then we have

- $$\mathbf{P}(X \mid \mathbf{d}) = \sum \mathbf{P}(X \mid \mathbf{d}, h_i) \mathbf{P}(h_i \mid \mathbf{d}) = \sum \mathbf{P}(X \mid h_i) P(h_i \mid \mathbf{d}) \,, \tag{20.2}$$

- where we have assumed that each hypothesis determines a probability distribution over X.
- This equation shows that predictions are weighted averages over the predictions of the individual hypotheses. The hypotheses themselves are essentially "intermediaries" between the raw data and the predictions. The key quantities in the Bayesian approach are the hypothesis prior, $P(h_i)$, and the likelihood of the data under each hypothesis, $P(d \mid h_i)$.
- For our candy example, we will assume for the time being that the prior distribution over h1, . . . , h5 is given by 0.1, 0.2, 0.4, 0.2, 0.1, as advertised by the manufacturer. The likelihood of the data is calculated under the assumption that the observations are i.i.d.

$$P(\mathbf{d} \mid h_i) = \prod_j P(d_j \mid h_i) \ .$$

- For example, suppose the bag is really an all-lime bag (h_5) and the first 10 candies are all lime; then $P(d \mid h_3)$ is 0.510, because half the candies in an h_3 bag are lime.2 Figure 20.1(a) shows how the posterior probabilities of the five hypotheses change as the sequence of 10 lime candies is observed.
- Notice that the probabilities start out at their prior values, so h_3 is initially the most likely choice and remains so after 1 lime candy is unwrapped. After 2 lime candies are unwrapped, h_4 is most likely; after 3 or more, h_5 (the dreaded all-lime bag) is the most likely. After 10 in a row, we are fairly certain of our fate. Figure 20.1(b) shows the predicted probability that the next candy is lime, based on Equation (20.2). As we would expect, it increases monotonically toward 1.

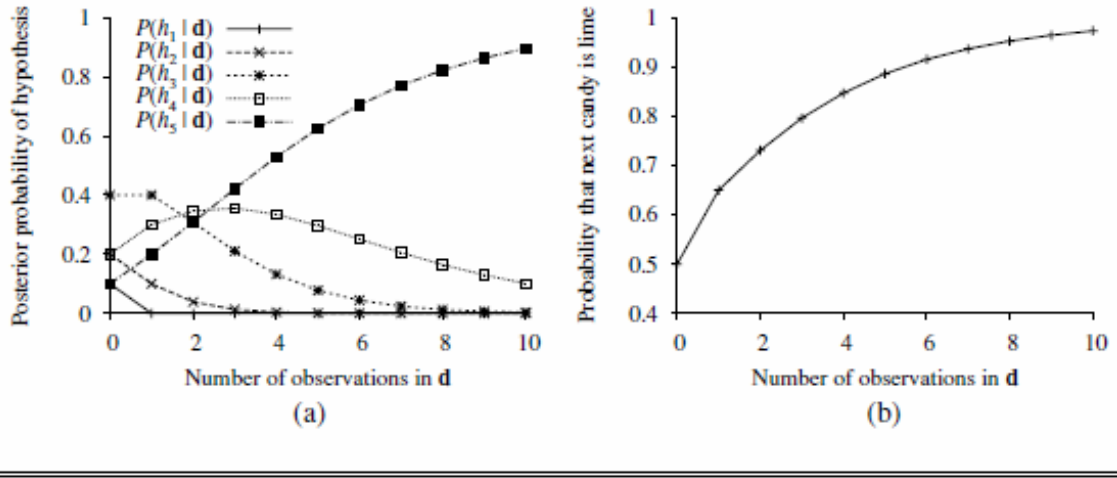


Figure 20.1 (a) Posterior probabilities $P(h_i \mid d_1, \ldots, d_N)$ from Equation (20.1). The number of observations N ranges from 1 to 10, and each observation is of a lime candy. (b) Bayesian prediction $P(d_{N+1} = lime \mid d_1, \ldots, d_N)$ from Equation (20.2).

5.7 Reinforcement Learning

- This passage introduces Reinforcement Learning (RL) — a method in which an agent learns through rewards rather than labeled examples.
- Unlike supervised learning (which provides the correct action for each situation), RL agents learn by interacting with an environment, receiving rewards (positive or negative feedback) based on their actions.
- For example, in chess, the agent might only get feedback at the end of the game (win or lose), while in other tasks like playing ping-pong or learning to walk, feedback is more frequent. The goal of RL is to learn an optimal policy—a way of acting that maximizes the expected total reward—without prior knowledge of how the environment works.
- RL can train systems for tasks that are too complex to program directly (e.g., playing games or flying helicopters).
- There are three main agent types in RL:
- **Utility-based agent** – learns a utility function for states and uses a model of the environment to select actions maximizing expected utility.
- **Q-learning agent** – learns a Q-function (utility of state–action pairs) and can make decisions without knowing the environment's model.
- **Reflex agent** – directly maps states to actions (learns a policy).
- The chapter then divides RL into key areas:

- Passive learning: Policy is fixed; the agent learns state utilities or environment models.
- Active learning: The agent explores actions to discover the best policy (exploration problem).
- Inductive learning and direct policy learning: Speed up learning or learn policies directly.

5.7.1 passive reinforcement learning

- In passive learning, the agent's policy π is fixed: in state s, it always executes the action π(s). Its goal is simply to learn how good the policy is—that is, to learn the utility function $U^{\pi}(s)$.
- The agent operates in a fully observable environment
- It starts from state (1,1) and continues until it reaches a terminal state ((4,2) or (4,3)), observing at each step:
- The current state,
- and The reward received in that state.
- Each trial is one complete run through the environment following π.
- From these experiences (sequences of states and rewards), the agent learns the expected total reward from each state under policy π.

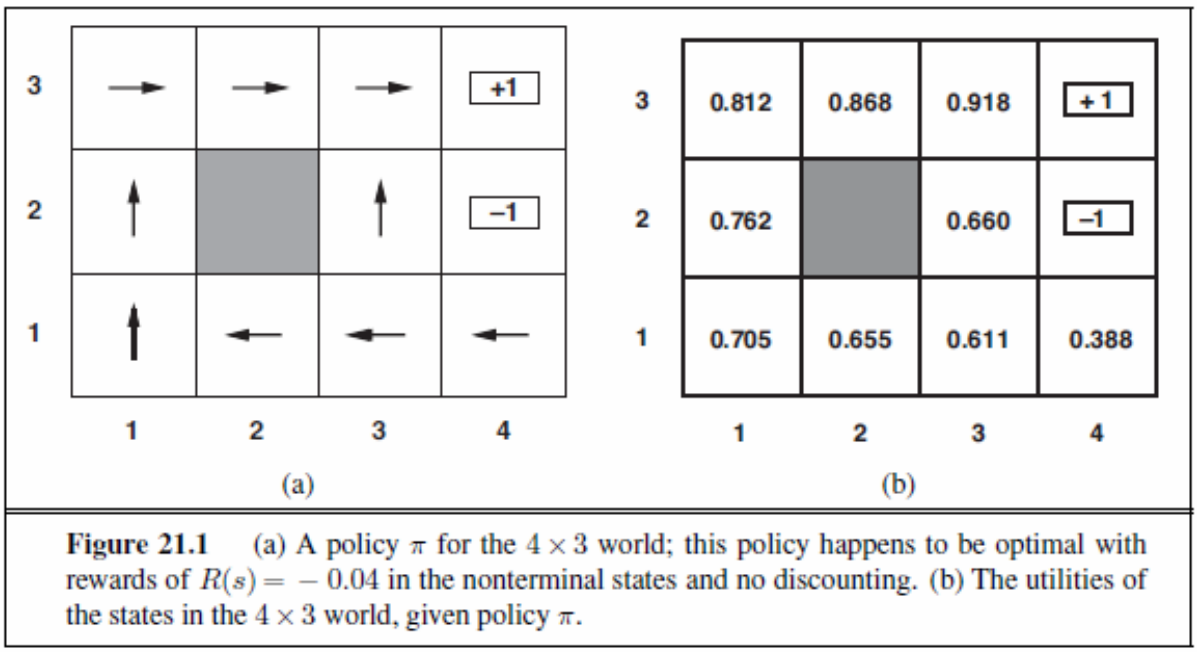


Figure 21.1 (a) A policy π for the 4×3 world; this policy happens to be optimal with rewards of $R(s) = -0.04$ in the nonterminal states and no discounting. (b) The utilities of the states in the 4×3 world, given policy π.

trials might look like this:

$$(1,1)_{-.04} \rightsquigarrow (1,2)_{-.04} \rightsquigarrow (1,3)_{-.04} \rightsquigarrow (1,2)_{-.04} \rightsquigarrow (1,3)_{-.04} \rightsquigarrow (2,3)_{-.04} \rightsquigarrow (3,3)_{-.04} \rightsquigarrow (4,3)_{+1}$$
$$(1,1)_{-.04} \rightsquigarrow (1,2)_{-.04} \rightsquigarrow (1,3)_{-.04} \rightsquigarrow (2,3)_{-.04} \rightsquigarrow (3,3)_{-.04} \rightsquigarrow (3,2)_{-.04} \rightsquigarrow (3,3)_{-.04} \rightsquigarrow (4,3)_{+1}$$
$$(1,1)_{-.04} \rightsquigarrow (2,1)_{-.04} \rightsquigarrow (3,1)_{-.04} \rightsquigarrow (3,2)_{-.04} \rightsquigarrow (4,2)_{-1} .$$

Step1:

- The agent starts at (1,1).
- Each move gives a small negative reward (-0.04) for taking a step (to discourage wandering).
- Finally, it reaches (4,3), a terminal goal state, receiving +1 reward (success).

Step2:

- Again starts at (1,1) and ends at (4,3).
- Slightly different path due to stochastic transitions (actions don't always go as planned).
- Same pattern: small penalties for steps and a +1 reward at the goal.

Step3:

Starts the same way but ends at (4,2), a negative terminal state with –1 reward (a failure or pitfall).

The agent still receives –0.04 for each intermediate step before the final penalty.

The **utility function** is defined as:

$$U^{\pi}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R(S_t)\right]$$

where:

:reward at time *t*,

: discount factor (set to 1 here),

:expected value over possible transitions.

5.7.1.1 Direct utility estimation

- Direct utility estimation is just an instance of supervised learning where each example has the state as input and the observed reward-to-go as output. This means that we have reduced reinforcement learning to a standard inductive learning problem.

- Direct utility estimation succeeds in reducing the reinforcement learning problem to an inductive learning problem, about which much is known.
- Unfortunately, it misses a very important source of information, namely, the fact that the utilities of states are not independent!
- The utility of each state equals its own reward plus the expected utility of its successor states. That is, the utility values obey the Bellman equations for a fixed policy.

$$U^{\pi}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi(s)) U^{\pi}(s')$$

- In reality, the utility of a state ( ) depends on:
  - ✓ The immediate reward ( ), and
  - ✓ The expected utility of its successor states.
- where:
- =immediate reward in state
- = discount factor ( )
- $P(s'|s,\pi(s))$ = transition probability to next state under policy

Why Direct Utility Estimation Falls Short

- Direct utility estimation does not enforce this Bellman relationship.
  As a result:
- It may require many samples to learn accurate utilities.
- It can produce inconsistent estimates, since nearby states' utilities might not agree with the dynamics of the environment.

```
function PASSIVE-ADP-AGENT(percept) returns an action
  inputs: percept, a percept indicating the current state s' and reward signal r'
  persistent: π, a fixed policy
              mdp, an MDP with model P, rewards R, discount γ
              U, a table of utilities, initially empty
              N_sa, a table of frequencies for state–action pairs, initially zero
              N_s'|sa, a table of outcome frequencies given state–action pairs, initially zero
              s, a, the previous state and action, initially null

  if s' is new then U[s'] ← r'; R[s'] ← r'
  if s is not null then
      increment N_sa[s, a] and N_s'|sa[s', s, a]
      for each t such that N_s'|sa[t, s, a] is nonzero do
          P(t | s, a) ← N_s'|sa[t, s, a] / N_sa[s, a]
  U ← POLICY-EVALUATION(π, U, mdp)
  if s'.TERMINAL? then s, a ← null else s, a ← s', π[s']
  return a
```

Figure 21.2 A passive reinforcement learning agent based on adaptive dynamic programming. The POLICY-EVALUATION function solves the fixed-policy Bellman equations, as described on page 657.

5.7.1.2 Adaptive dynamic programming

- An adaptive dynamic programming (or ADP) agent takes advantage of the constraints among the utilities of states by learning the transition model that connects them and solving the corresponding Markov decision process using a dynamic programming method.
- Adaptive Dynamic Programming agents exploit the relationships among state utilities by learning both:
  - ✓ The transition model ('| ,), and
  - ✓ The reward function ( ).
- Once these are learned, the agent can solve for the utilities ( ) using Dynamic Programming (DP) methods.

For Passive Learning Agents

For a passive agent (following a fixed policy):

- The agent plugs its learned ('| , ( )) and ( ) into the Bellman equation:

Because this is a linear system, it can be solved efficiently using:

- ✓ Linear algebra methods, or
- ✓ Modified Policy Iteration, a simplified form of Value Iteration, which updates utilities incrementally after each model update.

- Since the model changes only slightly with each observation, using the previous utilities as starting values allows for faster convergence.

- The process of learning the model itself is easy, because the environment is fully observable.
- This means that we have a supervised learning task where the input is a state–action pair and the output is the resulting state. In the simplest case, we can represent the transition model as a table of probabilities.
- We keep track of how often each action outcome occurs and estimate the transition probability P(s′|s,a) from the frequency with which s is reached when executing a in s.

Example:

If the action Right is executed three times in state (1,3), and it results in (2,3) twice, then:

$((2,3)|(1,3),\text{Right})=2/3$

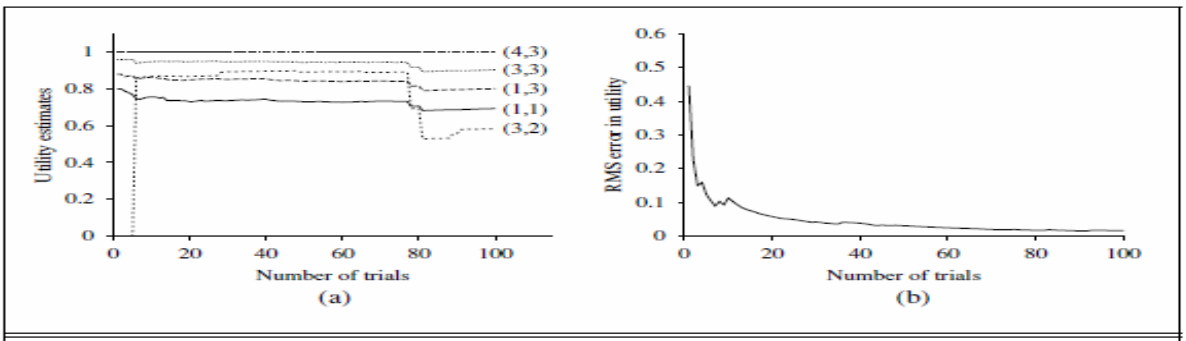


Figure 21.3 The passive ADP learning curves for the 4×3 world, given the optimal policy shown in Figure 21.1. (a) The utility estimates for a selected subset of states, as a function of the number of trials. Notice the large changes occurring around the 78th trial—this is the first time that the agent falls into the -1 terminal state at (4,2). (b) The root-mean-square error (see Appendix A) in the estimate for $U(1, 1)$, averaged over 20 runs of 100 trials each.

- ADP improves over direct utility estimation by learning a model of the environment and solving the Bellman equations using dynamic programming. This approach uses the structure of the MDP, leading to faster and more consistent learning.

Passive ADP Agent Overview

- The Passive ADP agent (as shown in Figure 21.2 of AIMA) works by:
  - ✓ Learning the transition model ('| ,) and the reward function ( ).
  - ✓ Using Dynamic Programming to compute utilities ( ) via the Bellman equations.
  - ✓ Following a fixed policy (it doesn't try to improve the policy — hence "passive").
- Its performance (Figure 21.3) on the 4×3 grid world shows that its value estimates improve rapidly, limited mainly by how fast it learns the transition model.

Advantages and Limitations

Advantage: ADP provides a benchmark for other reinforcement learning algorithms

— its learning speed depends only on model estimation accuracy.

Limitation: It becomes computationally intractable for large state spaces.

Example: In backgammon, there are about 10^{50} possible states — meaning the Bellman equations would involve solving 10^{50} simultaneous equations.

Connection to Bayesian Learning

- The ADP algorithm uses maximum-likelihood estimation (MLE) to learn the model ('| ,). However, this approach assumes that the estimated model is exactly correct, which can lead to overconfident and risky behavior.
- Example: A taxi agent that ignores a red light a few times without consequence might conclude (incorrectly) that red lights are harmless — and form a dangerous policy.

Towards Bayesian Reinforcement Learning

To handle model uncertainty, the agent can adopt a Bayesian approach:

- Assume a prior probability ( ) for each possible hypothesis (model) .
- Update to a posterior probability (|) using Bayes' rule as new experience is observed. Compute the expected utility for each policy , averaged over all plausible models:

$$\pi^* = \underset{\pi}{\operatorname{argmax}} \sum_h P(h \mid \mathbf{e}) u_h^\pi .$$

where is the expected utility of policy under model .

- This yields a policy that performs well across all likely models, not just the single most likely one — making it more robust and safer.

When the Agent Keeps Learning

- If the agent continues to learn (not just act on a fixed model), then finding the optimal policy becomes much more complex because:
  - ✓ The agent's future experiences will change its beliefs about the transition model .
  - ✓ Thus, its current decision affects both immediate rewards and what it will learn later.
- This turns the problem into a Partially Observable Markov Decision Process (POMDP), where:
  - ✓ The belief state represents a probability distribution over possible models, not over environment states.

- ✓ Solving this “learning POMDP” analytically is extremely hard.

- This theoretical view provides the foundation for the exploration problem — balancing exploration (learning better models) and exploitation (using what is already known).

Robust Control

An alternative to Bayesian reasoning is drawn from Robust Control Theory, where we assume:

- There’s a set of possible models (not necessarily with probabilities).
- We want a policy that performs well in the worst-case scenario over that set.

The robust optimal policy is defined as:

$$\pi^* = \operatorname*{argmax}_{\pi} \min_{h} u_h^{\pi} .$$

where:

- is the expected utility of policy under model .
- The inner “min” finds the worst-case performance for each policy.
- The outer “max” chooses the policy that performs best among those worst cases.

This ensures the agent is safe and conservative — it avoids catastrophic outcomes even if the true model differs from the learned one.

5.7.1.3 Temporal-difference learning

Instead of explicitly solving the Bellman equations (as ADP does), TD learning updates utility estimates directly from observed transitions between states.

When a transition from state to is observed, the utility of is updated as:

where:

= learning rate

=reward received in

= discount factor

=current utility estimate of the successor state

This equation is called the Temporal-Difference (TD) update rule, because it uses the difference between successive utilities (the “temporal difference”) to make incremental corrections.

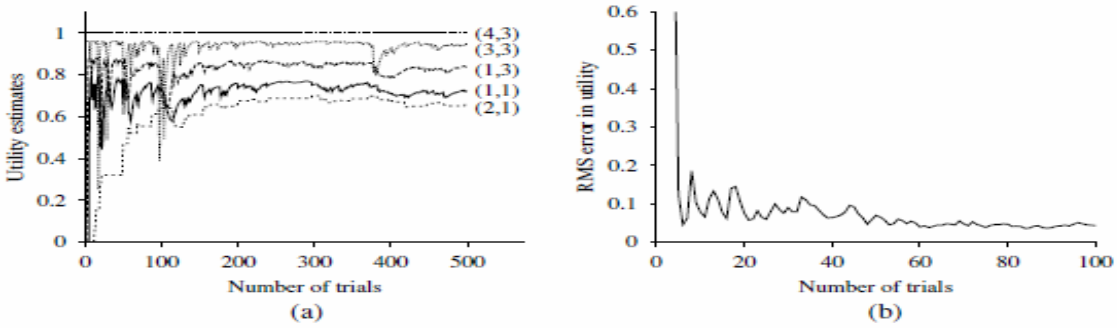


Figure 21.5 The TD learning curves for the 4×3 world. (a) The utility estimates for a selected subset of states, as a function of the number of trials. (b) The root-mean-square error in the estimate for $U(1, 1)$, averaged over 20 runs of 500 trials each. Only the first 100 trials are shown to enable comparison with Figure 21.3.

How TD Learning Works

- TD learning adjusts utilities locally so that each state's value agrees with its observed successor.
- Over time, these local corrections drive the estimates toward satisfying the Bellman equations.
- If decreases as a state is visited more often, the utility estimates converge to the correct values.

Example: If (1,3)=0.84 and (2,3)=0.92, and moving from (1,3) → (2,3) gives reward –0.04,then the expected equilibrium is 0.880.88, suggesting (1,3)'s utility should increase slightly.

Performance and Comparison with ADP

TD Learning:

- ✓ Simpler and less computationally expensive
- ✓ Learns without a transition model
- ✓ Shows more variability and slightly slower convergence

ADP:

- ✓ Requires learning a transition model
- ✓ Updates utilities to agree with all possible successors, not just the observed one
- ✓ Can make multiple updates per observation to restore consistency between and

Both methods seek consistency with the Bellman equation —TD does it sample by sample, while ADP does it model-wide.

Relationship Between TD and ADP

- When averaged over many transitions, TD's updates approximate ADP's weighted updates.
- ADP can be seen as performing many "imagined" or simulated transitions —TD can imitate this idea by generating pseudo-experiences from its learned model.

This bridges the gap between the two methods.

Approximate ADP and Prioritized Sweeping

For very large state spaces, full ADP (which solves large linear systems) is intractable.

Approximate ADP algorithms address this by:

- Limiting the number of updates after each observation.
- Focusing updates on the most important states using prioritized sweeping —a heuristic that updates states whose successors recently changed significantly in utility.

This makes learning as fast as full ADP (in terms of data efficiency) but far more computationally efficient.

5.7.2 Active reinforcement learning

- A passive learning agent has a fixed policy that determines its behavior. An active agent must decide what actions to take. Let us begin with the adaptive dynamic programming agent and consider how it must be modified to handle this new freedom.
- First, the agent will need to learn a complete model with outcome probabilities for all actions, rather than just the model for the fixed policy. The simple learning mechanism used by PASSIVE-ADP-AGENT will do just fine for this.
- Next, we need to take into account the fact that the agent has a choice of actions. The utilities it needs to learn are those defined by the optimal policy; they obey the Bellman equations

$$U(s) = R(s) + \gamma \max_a \sum_{s'} P(s' \mid s, a) U(s') .$$

- These equations can be solved to obtain the utility function U using the value iteration or policy iteration algorithms
- Having obtained a utility function U that is optimal for the learned model, the agent can extract an optimal action by one-step look-ahead to maximize the expected utility; alternatively, if it uses policy iteration, the optimal policy is already available, so it should simply execute the action the optimal policy recommends.

5.7.2.1 Exploration

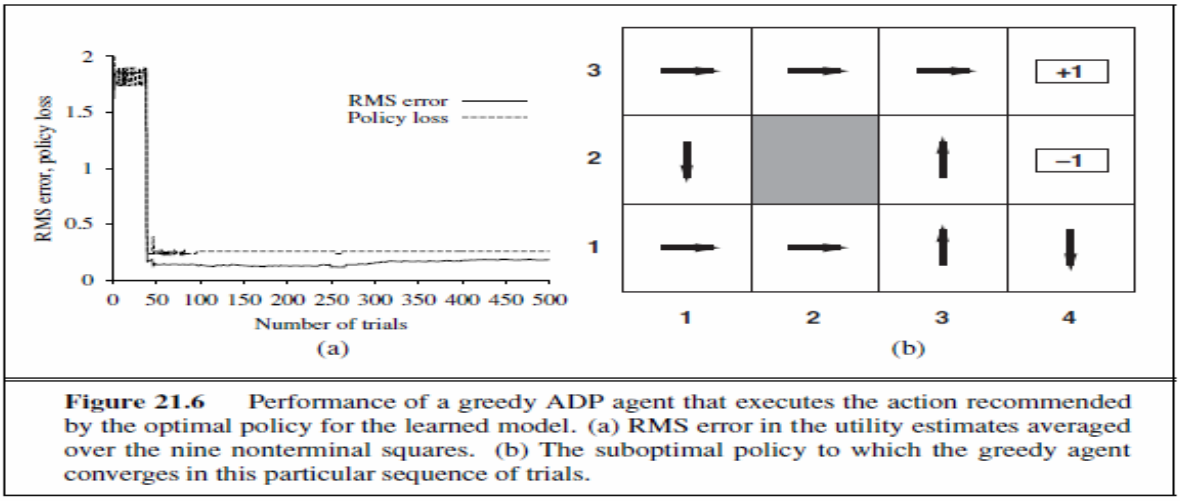


Figure 21.6 Performance of a greedy ADP agent that executes the action recommended by the optimal policy for the learned model. (a) RMS error in the utility estimates averaged over the nine nonterminal squares. (b) The suboptimal policy to which the greedy agent converges in this particular sequence of trials.

- The agent eventually discovers a suboptimal route (via states (2,1) → (3,1) → (3,2) → (3,3)) that yields a small reward.
- Once it finds this, it sticks to that route from the 276th trial onward.
- It never explores other routes like (1,2) → (1,3) → (2,3), which actually lead to the best reward.
- Hence, it gets stuck in a local optimum, failing to learn the true utilities or the globally optimal policy.

Why "Choosing the Optimal Action" Can Be Wrong

- The learned model ≠ true environment.
- What appears "optimal" in the current model may be suboptimal in reality.
- The agent's lack of exploration prevents it from correcting its mistaken beliefs about the environment.
- Exploration vs. Exploitation Tradeoff

Exploitation → Acting according to the current best knowledge to maximize immediate reward.

Exploration → Trying new or uncertain actions to improve the model and gain better rewards in the future.

- Too much exploitation → Agent gets stuck in a rut.
- Too much exploration → Agent wastes time experimenting without using what it's learned.
- The key is to balance both.

GLIE — "Greedy in the Limit of Infinite Exploration"

- To guarantee optimal long-term learning, an exploration strategy should satisfy GLIE conditions:

1. Infinite exploration:

  - ✓ Every action in every state must be tried infinitely often,
  - ✓ ensures no optimal action is missed by chance.

2. Greedy in the limit:

  - ✓ As time goes on, exploration gradually decreases,
  - ✓ so the policy eventually becomes fully greedy (optimal) with respect to the true model.

Example

- A simple GLIE method:
  - ✓ At trial : choose a random action with probability 1/ , otherwise follow greedy policy.
  - ✓ Guarantees convergence to the optimal policy
  - ✓ But very slow, since exploration decreases only gradually.
- A better approach: Prioritize exploring rarely tried actions, Avoid actions known to have low utility —this can be implemented by modifying the Bellman equation to temporarily assign higher utility to underexplored actions.

In optimistic exploration, the agent assumes unexplored actions might yield great rewards, encouraging it to try them. It uses an optimistic utility estimate $^{+}(\ )$, updated with an exploration function $(\ ,\ )$ that balances greed (favoring high-utility actions) and curiosity (favoring less-tried actions).A simple exploration function is:

$$f(u,n) = \begin{cases} R^{+} & \text{if } n < N_e \\ u & \text{otherwise} \end{cases}$$

- where R^{+} is an optimistic estimate of the best possible reward obtainable in any state and N_e is a fixed parameter. This will have the effect of making the agent try each action–state pair at least N_e times.
- Using $^{+}$ in updates allows the agent to favor unexplored areas indirectly—spreading optimism backward through the state space. This leads to rapid learning and near-optimal performance

5.7.2.2 Learning an action-utility function

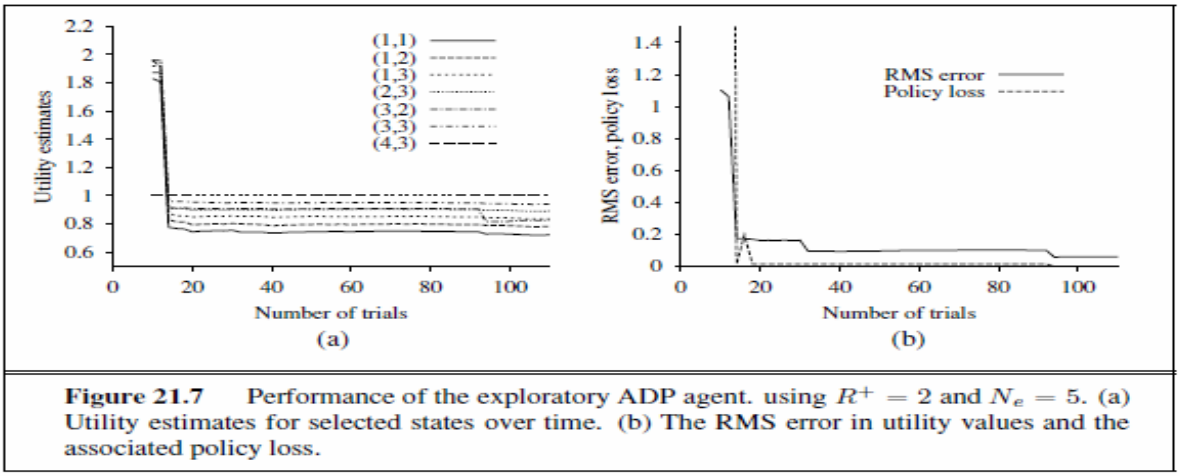


Figure 21.7 Performance of the exploratory ADP agent. using $R^+ = 2$ and $N_e = 5$. (a) Utility estimates for selected states over time. (b) The RMS error in utility values and the associated policy loss.

In an **active TD learning agent**, unlike the passive case, the agent must **choose actions** while learning. It still uses the **same TD update rule** as before:

Even though random outcomes might mislead the agent occasionally, over many trials the updates naturally average out according to the true transition probabilities—ensuring convergence to correct values, just like ADP.

A major advancement is **Q-learning**, which learns **action utilities** instead of state utilities . They are related by:

$$U(s) = \max_a Q(s, a)$$

The **Q-function** captures the value of performing action in state and is governed by:

$$Q(s,a) = R(s) + \gamma \sum_{s'} P(s' \mid s, a) \max_{a'} Q(s', a') .$$

Unlike ADP, **Q-learning is model-free** — it does not need to know or learn transition probabilities $P(s' \mid s, a)$ Instead, it updates directly from experience, making it simpler and more efficient for complex environments.

function Q-LEARNING-AGENT(*percept*) **returns** an action
  inputs: *percept*, a percept indicating the current state s' and reward signal r'
  persistent: Q, a table of action values indexed by state and action, initially zero
    N_{sa}, a table of frequencies for state–action pairs, initially zero
    s, a, r, the previous state, action, and reward, initially null

  if TERMINAL?(s) **then** $Q[s, None] \leftarrow r'$
  if s is not null **then**
    increment $N_{sa}[s, a]$
    $Q[s, a] \leftarrow Q[s, a] + \alpha(N_{sa}[s, a])(r + \gamma \max_{a'} Q[s', a'] - Q[s, a])$
  $s, a, r \leftarrow s', \text{argmax}_{a'} f(Q[s', a'], N_{sa}[s', a']), r'$
  return a

Figure 21.8 An exploratory Q-learning agent. It is an active learner that learns the value $Q(s, a)$ of each action in each situation. It uses the same exploration function f as the exploratory ADP agent, but avoids having to learn the transition model because the Q-value of a state can be related directly to those of its neighbors.

- Q-learning updates the value of each state–action pair using the rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha(R(s) + \gamma \max_{a'} Q(s',a') - Q(s,a)) ,$$

This happens whenever the agent performs action in state and transitions to .It uses an **exploration function** (like in ADP) or a simpler random exploration strategy that decreases over time.

A related algorithm, **SARSA** (*State-Action-Reward-State-Action*), uses the update rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha(R(s) + \gamma\, Q(s',a') - Q(s,a)) ,$$

The key difference is that **Q-learning is off-policy**—it learns from the *best possible* next action, regardless of what the agent actually did—while **SARSA is on-policy**, updating based on the *action actually taken*.

Both methods can learn the optimal policy (like in the 4×3 world) but generally learn **slower than ADP**, since they don't maintain a full model to enforce global consistency among Q-values.